PROJECT REPORT

on

AUTONOMOUS PATH PLANNING FOR WAREHOUSE ROBOTS

Submitted by

SASIKUMAR.S

Register Number: 711525BCS152

in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING

KALAIGNAR KARUNANIDHI INSTITUTE OF TECHNOLOGY

(KIT-CBE), Coimbatore

Affiliated to Anna University, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2026–2027

BONAFIDE CERTIFICATE

Certified that this project report titled “AUTONOMOUS PATH PLANNING FOR WAREHOUSE ROBOTS” is the bonafide work of SASIKUMAR.S (Register Number: 711525BCS152), who carried out the project work under my supervision in the Department of Computer Science and Engineering, Kalaignar Karunanidhi Institute of Technology (KIT-CBE), Anna University, Chennai, during the academic year 2026–2027.

Signature of Project Guide
Sumathy

Signature of Head of the Department

DECLARATION

I, SASIKUMAR.S(Register Number: 711525BCS152), hereby declare that the project report titled “AUTONOMOUS PATH PLANNING FOR WAREHOUSE ROBOTS”, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering, is a record of the original work carried out by me under the guidance of Sumathy, Department of Computer Science and Engineering, Kalaignar Karunanidhi Institute of Technology (KIT-CBE), Anna University, Chennai.

I further declare that this work has not been submitted, in part or in full, to any other university or institution for the award of any degree, diploma, or other similar title, and that all sources of information and literature used in this report have been duly acknowledged.

Signature of the
Student
(711525BCS152)
SASIKUMAR.S

ACKNOWLEDGEMENT

I express my sincere gratitude to the Management and the Principal of Kalaignar Karunanidhi Institute of Technology (KIT-CBE) for providing the computing facilities, infrastructure and academic environment needed to undertake and complete this project successfully.

I extend my deep gratitude to the Head of the Department of Computer Science and Engineering for the continuous encouragement and guidance provided throughout the design and implementation phases.

I express my heartfelt gratitude to my Project Guide, Sumathy, for the invaluable suggestions, critical reviews and patient guidance extended at every milestone of algorithm design, implementation and documentation.

I also thank the faculty members and technical staff of the Department of Computer Science and Engineering for their cooperation and constructive feedback during the review presentations. Finally, I thank my parents, family members and friends for their constant moral support and encouragement throughout my engineering studies.

ABSTRACT

Warehouses and fulfilment centres increasingly rely on fleets of autonomous mobile robots to move goods between storage racks and packing docks. The efficiency of such a fleet depends on two coupled problems: finding a short, safe route for each robot through a cluttered, aisle-based floor plan, and ensuring that several robots sharing the same aisles never collide or deadlock. Naive solutions plan each robot independently with a shortest-path algorithm; this produces cost-optimal individual routes but, as this project shows, frequently yields robots that occupy the same cell or swap positions in the same time step.

This project presents the design, implementation and empirical evaluation of an ***Autonomous Path Planning system for Warehouse Robots***. The warehouse is modelled as a four-connected weighted grid in which shelf cells are obstacles and congested zones (such as the packing dock) carry a higher traversal cost. Four single-robot planners are implemented in Java 21 and compared: Breadth-First Search (BFS), Dijkstra's algorithm, A* search with an admissible Manhattan-distance heuristic, and Greedy Best-First Search. For fleets of robots, a ***prioritised space-time A**** planner with a vertex/edge reservation table is implemented so that every robot receives a collision-free plan in the time dimension. The system is exposed through a browser-based interface served from localhost:8080 that visualises the warehouse and three robots.

The planners were evaluated on 500 random start–goal queries on each of three warehouse sizes (20×15, 40×30 and 80×60 cells) and on 300 random three-robot and six-robot scenarios. A* returned the optimal-cost path in 100% of the 1,500 queries while expanding 66.8%, 76.9% and 82.7% fewer nodes than Dijkstra on the three sizes. BFS, which ignores cell costs, was non-optimal on up to 14.2% of queries, and Greedy Best-First on up to 16.2%. In multi-robot trials, independent planning produced at least one vertex or swap conflict in 25.3% of three-robot scenarios (76 of 300) on both the 20×15 and 40×30 floors, and in 76.3% of six-robot scenarios on the 20×15 floor. Prioritised space-time A* solved all 900 scenarios with zero residual conflicts at an average cost increase of only 1.3–3.9% in total travel steps.

Keywords: Warehouse Robots, Path Planning, A* Search, Dijkstra's Algorithm, Multi-Agent Path Finding, Space-Time A*, Reservation Table, Grid Map, Java.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Introduction 1
1.2 Background of the Project 1
1.3 Problem Statement 1
1.4 Motivation and Scope 1
1.5 Project Objectives 1
1.6 Organization of the Report 1
1.7 Significance of the Project 1
1.8 Applications of the System 1
1.9 Assumptions and Constraints 2
CHAPTER 2: LITERATURE REVIEW 1
2.1 Introduction 2
2.2 Robotic Warehousing Systems 1
2.3 Single-Robot Path Planning 2
2.4 Multi-Robot Path Finding 1
2.5 Comparative Analysis of Planning 2
Approaches 2.6 Research Gap and Motivation 1
2.7 Review of Core Technologies 3
2.8 Summary of Related Work 1
2.9 Chapter Summary 3
CHAPTER 3: SYSTEM ANALYSIS AND ARCHITECTURE 1
3.1 Existing System and Limitations 4
3.2 Proposed System 1
3.3 Functional Requirements 4
3.4 Non-Functional Requirements 1
3.5 Overall System Architecture 4
3.6 Planning Workflow 1
3.7 Module Description 4
3.8 Use Case Analysis 1
3.9 Data Model and Class Structure 4
3.10 Data Flow Modeling 1
3.11 Feasibility Study 4
3.12 Data Dictionary 1
3.13 Sequence of Operations 5
3.14 Use Case Specifications 1
3.15 Class Description 5
CHAPTER 4: MATERIALS AND METHODS 1
4.1 Development Tools and Technologies 6
4.2 Warehouse Environment Model 1
4.3 Cost Model and Heuristic 6
4.4 Single-Robot Planners 1
7
1
7
1
7
1

4.5 Admissibility and Optimality of A* 2
4.6 Multi-Robot Coordination: Space-Time 6
A* 4.7 Reservation Table and Conflict 2
Definitions 4.8 Web Interface and 6
Simulation 2
4.9 Complexity Analysis 6
4.10 Layout Generation Algorithm 2
4.11 Data Structures and Tie-Breaking 7
4.12 Priority Ordering Strategy 2
4.13 Wait Actions and Goal Reservation 7
4.14 Deadlock and Livelock Considerations 2
4.15 Verification Methodology 7
CHAPTER 5: EXPERIMENTAL SETUP AND PROCEDURE
5.1 Hardware and Software Environment 2
5.2 Reproducing the Experiments 7
5.3 Single-Robot Experiment Design 2
5.4 Multi-Robot Experiment Design 7
5.5 Running the Web Application 2
5.6 Test Case Design 8
5.7 Evaluation Metrics 2
5.8 Performance Test Procedure 8
5.9 Test Tools and Acceptance Criteria 2
CHAPTER 6: RESULTS AND DISCUSSION 8
6.1 Application Interface Overview 2
6.2 Warehouse Layout 9
6.3 Single-Robot Planner Results 2
6.4 Search Behaviour Visualisation 9
6.5 Multi-Robot Results 2
6.6 Discussion of Results 9
6.7 Limitations 2
6.8 Comparison with Existing Approaches 9
6.9 Sample System Outputs 2
6.10 Error Analysis and Lessons Learned 9
CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENT 3
7.1 Conclusion 0
7.2 Project Contributions 3
7.3 Mapping of Objectives to Achieved Outcomes 0
7.4 Future Enhancements 3
7.5 Social and Economic Impact 0
7.6 Challenges Faced and Solutions 3
7.7 Final Remarks 1
REFERENCES 3
APPENDIX A: Installation and Execution Guide 1
APPENDIX B: Important Parameters and Constants 3
APPENDIX C: Best-First Search Routine (excerpt) 2
3
2
3
2
3

APPENDIX D: Space-Time A* (excerpt) 4
APPENDIX E: Sample Benchmark Output 3
APPENDIX F: Data Model Excerpt 4
APPENDIX G: Reservation and Conflict Check (excerpt) 4
APPENDIX H: Layout and Scenario Details 4
APPENDIX I: Project Folder Structure 5
APPENDIX J: User Manual 4
APPENDIX K: Glossary 6
APPENDIX L: Complete Source Code (WebWarehousePlanner.java) 4
7
4
8
4
9
5
0
5
1

LIST OF FIGURES & TABLES

List of Figures

Figure 3.1: Layered Architecture of the Warehouse Path Planner 1
Figure 3.2: Multi-Robot Planning Pipeline 8
Figure 3.3: Use Case Diagram for Operator, Robot and Evaluator Roles 1
Figure 3.4: Main Classes and Routines of the Planner 9
Figure 3.5: Level 0 Data Flow Diagram (Context Level) 2
Figure 3.6: Sequence Diagram for Fleet Planning 0
Figure 6.1: Web Interface Screenshot (to be inserted) 2
Figure 6.2: Synthetic Warehouse Layout (40 × 30) 0
Figure 6.3: Nodes Expanded and Optimality Rate Across Warehouse Sizes 2
Figure 6.4: Dijkstra vs A* vs Greedy Search Behaviour on One Query 1
Figure 6.5: Independent vs Prioritised Space-Time Planning for Three Robots 2

List of Tables

Table 2.1: Comparison of Path Planning Approaches 1
Table 2.2: Summary of Related Work 4
Table 3.1: Non-Functional Requirements 1
Table 3.2: Qualitative Comparison of Deployment 6
Approaches Table 3.3: Data Dictionary – Grid 1
Table 3.4: Data Dictionary – Search Result 7
Table 3.5: Data Dictionary – Reservation Structures 2
Table 3.6: Use Case Specification – Plan a Single Route 1
Table 3.7: Use Case Specification – Plan Routes for the Fleet 2
Table 3.8: Use Case Specification – Run the Benchmark 1
Table 3.9: Use Case Specification – View Route and Metrics 2
Table 3.10: Main Classes and Responsibilities 1
Table 4.1: Development Stack 2
Table 4.2: Cell Types and Traversal Costs 2
Table 4.3: Data Structures Used 2
Table 5.1: Experimental Configuration 2
Table 5.2: System Verification Test Cases 2
Table 5.3: Evaluation Metrics 3
Table 5.4: Tools Used for Testing 2
Table 5.5: Acceptance Criteria and Measured Values 3
Table 6.1: Single-Robot Planner Results (500 queries per 2
size) Table 6.2: Multi-Robot Planning Results (300 scenarios 3
per row) Table 6.3: Relative Improvements Derived from 2
Measured Results Table 6.4: Comparison of this Work with 3
Existing Approaches Table 7.1: Mapping of Objectives to 2
Outcomes 5
Table 7.2: Challenges and Solutions 2
5
2
7
2
9

LIST OF ABBREVIATIONS

Abbreviation	*Full Expansion / Meaning*
A*	A-star search algorithm (f = g + h)
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
API	Application Programming Interface
BFS	Breadth-First Search
CBS	Conflict-Based Search
CSS	Cascading Style Sheets
D* Lite	Dynamic A* Lite (incremental replanning)
FIFO	First In, First Out
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JDK	Java Development Kit
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
MAPF	Multi-Agent Path Finding
PQ	Priority Queue
SOC	Sum of Costs (total travel steps of all robots)
UI	User Interface
WMS	Warehouse Management System

CHAPTER 1: INTRODUCTION

1.1 Introduction

E-commerce growth has transformed warehouses from passive storage buildings into high-throughput logistics systems. Items must be picked from racks, carried to packing or dispatch docks and returned within strict time windows. Fleets of autonomous mobile robots (AMRs) are now widely used for this purpose because they scale more flexibly than fixed conveyors and can be redeployed as the layout changes.

At the heart of every such fleet is a path planner. Given a map of the warehouse, a robot's current position and a destination, the planner must return a route that is short, avoids shelving and other obstacles and, when several robots share the floor, does not interfere with the routes of the others. A poor planner wastes travel time, causes congestion in narrow aisles and, in the worst case,
deadlocks the fleet. This project, ***Autonomous Path Planning for Warehouse Robots***, studies and implements planners that address these issues in a transparent and measurable way.

1.2 Background of the Project

A typical rack-based warehouse is highly structured: long parallel shelving blocks separated by narrow aisles, with cross-aisles at regular intervals and one or more packing docks near the edge. This structure is naturally represented as a grid in which each cell is either free or blocked. Graph-search algorithms such as Breadth-First Search, Dijkstra's algorithm (Dijkstra, 1959) and A* (Hart, Nilsson and Raphael, 1968) have well-understood optimality and complexity properties on such grids.

When several robots operate simultaneously the problem becomes Multi-Agent Path Finding (MAPF). Optimal MAPF is NP-hard in general, so practical systems use decoupled approaches such as prioritised planning with space-time reservations (Silver, 2005) or bounded-suboptimal search (Stern et al., 2019). Large commercial deployments such as the Kiva system described by Wurman, D'Andrea and Mountz (2008) coordinate hundreds of robots using centrally managed reservations of floor space and time.

1.3 Problem Statement

Warehouse robot navigation exhibits four operational difficulties that this project addresses:

- ***Cost-blind routing:*** Shortest-hop planners such as BFS ignore that some cells (the dock, narrow aisles, congested junctions) are more expensive to traverse, and so return routes that are not actually the cheapest.
- ***Search inefficiency:*** Uninformed search such as Dijkstra expands a large fraction of the floor for every query, which becomes wasteful as warehouses grow to thousands of cells.
- ***Inter-robot conflicts:*** When each robot is planned independently, two robots may occupy the same cell at the same time (a vertex conflict) or exchange cells head-on in an aisle (a swap conflict).

- ***Lack of transparent evaluation:*** Many demonstrations show a single path without quantifying optimality, search effort or conflict frequency over many random scenarios.

1.4 Motivation and Scope

The motivation of this project is to build an academically rigorous and reproducible prototype showing how classical artificial intelligence search techniques, combined with a simple reservation mechanism, can deliver optimal single-robot routes and conflict-free multi-robot plans in an aisle-based warehouse.

The scope covers: (i) a weighted grid model of a rack-based warehouse; (ii) implementation of BFS, Dijkstra, A* and Greedy Best-First planners in Java 21; (iii) a prioritised space-time A* planner for fleets of robots; (iv) a browser-based interface served on localhost:8080 that displays the warehouse and three robots; and (v) a benchmark harness that measures path cost, nodes expanded, optimality rate and conflict counts over randomised scenarios. The warehouse layout, robot kinematics and task arrivals are simulated; no physical robots or live sensor data are used.

1.5 Project Objectives

- ***Objective 1:*** To model a rack-based warehouse as a four-connected weighted grid with shelf obstacles and congested high-cost zones.
- ***Objective 2:*** To implement and compare BFS, Dijkstra, A* and Greedy Best-First search in terms of path cost, nodes expanded and optimality.
- ***Objective 3:*** To verify empirically that A* with a Manhattan-distance heuristic returns the same optimal cost as Dijkstra while expanding fewer nodes.
- ***Objective 4:*** To develop a prioritised space-time A* planner that produces vertex- and swap-conflict-free plans for three or more robots.
- ***Objective 5:*** To provide a browser-based visualisation of the warehouse and its three robots, together with a reproducible benchmark harness.

1.6 Organization of the Report

Chapter 1 introduces the background, problem, scope and objectives. ***Chapter 2*** reviews the relevant literature on robotic warehousing, single-robot search and multi-agent path finding. ***Chapter 3*** presents the system analysis, requirements and architecture. ***Chapter 4*** describes the environment model, algorithms and their properties. ***Chapter 5*** documents the experimental setup and test cases. ***Chapter 6*** presents results with figures and tables and discusses limitations. ***Chapter 7*** concludes and outlines future work. Appendices list parameters, pseudocode, code excerpts and raw benchmark output.

1.7 Significance of the Project

Path planning is a core capability of every autonomous mobile robot, and it is taught and evaluated mostly through isolated algorithm exercises. This project is significant because it places the algorithms in a realistic context, an aisle-based warehouse with congested zones and several

robots, and measures them with reproducible statistics rather than single examples. For a student, it connects the theory of graph search (optimality, admissibility, complexity) to engineering concerns such as collision avoidance and verification. For a warehouse operator, it demonstrates that an inexpensive, open and explainable planner can already remove most conflicts in small fleets.

1.8 Applications of the System

- ***E-commerce fulfilment centres:*** routing goods-to-person robots between racks and packing docks.
- ***Manufacturing intralogistics:*** moving parts between stores and assembly cells along fixed aisles.
- ***Cold-storage and pharmaceutical stores:*** where short, predictable routes reduce time in controlled environments.
- ***Teaching and research:*** a compact, reproducible testbed for comparing single-robot and multi-robot planners.
- ***Simulation before deployment:*** evaluating a proposed rack layout by measuring path cost and conflict frequency.

1.9 Assumptions and Constraints

- The warehouse map is known in advance and static during planning.
- Robots move one cell per time step in four directions; all robots have the same speed.
- Each robot has one start cell and one goal cell, and all start and goal cells in a scenario are distinct.
- Robots are treated as point-sized; turning cost, acceleration and robot footprint are not modelled.
- A robot that reaches its goal remains there and is treated as an obstacle for later planning.
- The system is a simulation prototype; no physical robots, sensors or wireless communication are used.
- Planning is centralised: one planner computes the plans of all robots.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Path planning for mobile robots lies at the intersection of graph theory, heuristic search and multi-agent coordination. This chapter reviews the literature that grounds the present work: robotic warehousing systems, classical single-robot search algorithms, and multi-agent path finding.

2.2 Robotic Warehousing Systems

Wurman, D'Andrea and Mountz [3] described the Kiva mobile-fulfilment system, in which hundreds of robots carry inventory pods to stationary pick stations. Their work showed that centralised, reservation-based coordination of a grid-structured floor can sustain high throughput and avoid deadlock. Their system motivates the grid-and-reservation abstraction adopted in this project, although the commercial system employs additional components such as task allocation, pod selection and congestion management that are outside the present scope.

2.3 Single-Robot Path Planning

Dijkstra's algorithm [1] computes the minimum-cost path in a graph with non-negative edge weights by expanding nodes in increasing order of accumulated cost g(n). It is optimal but uninformed, so it expands nodes in all directions. Hart, Nilsson and Raphael [2] introduced A*, which orders expansion by f(n) = g(n) + h(n), where h(n) estimates the remaining cost to the goal. If h never overestimates the true remaining cost (admissibility) A* is guaranteed to return an optimal path, and if h is also consistent each node is expanded at most once. On four-connected grids the Manhattan distance, scaled by the minimum edge cost, is both admissible and consistent. Greedy Best-First Search orders expansion by h(n) alone. It is fast but gives no optimality guarantee. For dynamic environments, incremental algorithms such as D* Lite [6] reuse previous search effort when the map changes; these are discussed as future work in this report.

2.4 Multi-Robot Path Finding

Erdmann and Lozano-Pérez [7] introduced prioritised planning, in which robots are planned one at a time and each treats the already-planned robots as moving obstacles. Silver [4] showed that planning in a space-time graph with a reservation table (Cooperative A*) gives a practical and fast method for conflict-free planning in grid worlds. Prioritised planning is not complete: a poor priority order can make a later robot unsolvable even when a joint solution exists. Sharon et al. [5] proposed Conflict-Based Search (CBS), which is optimal and complete but has exponential worst-case behaviour; Stern et al. [8] defined standard MAPF definitions, variants and benchmarks, and summarised the trade-offs between optimal, bounded-suboptimal and decoupled methods.

2.5 Comparative Analysis of Planning Approaches

Table 2.1: Comparison of Path Planning Approaches

Approach	*Optimal cost?*	*Uses heuristic?*	*Handles other robots?*	*Typical use*
BFS	Only for unit costs	No	No	Baseline, unweighted maps
Dijkstra [1]	Yes	No	No	Weighted maps, reference optimum
A* [2]	Yes (admissible h)	Yes	No	Fast optimal single-robot routing
Greedy Best-First	No	Yes (h only)	No	Very fast, approximate routing
D* Lite [6]	Yes	Yes	No (replans on map change)	Changing maps / sensing
Prioritised space-time A* [4],[7]	Per robot, given reservations	Yes	Yes (reservations)	Fast fleet planning, incomplete
CBS [5]	Optimal sum of costs	Low-level A*	Yes	Optimal MAPF, costly for many robots
This project	A* optimal; fleet conflict-free	Manhattan	Yes (vertex + swap)	Warehouse, 3 to 6 robots

2.6 Research Gap and Motivation

Published work reports the properties of each algorithm in isolation, but a student-scale, fully reproducible comparison that combines weighted-cost single-robot planning with conflict-free multi-robot planning on the same warehouse model, and that quantifies how often naive independent planning fails, is less commonly presented. This project fills that gap by implementing both layers in one Java code base and measuring them over randomised scenarios with fixed seeds.

2.7 Review of Core Technologies

Grid maps. An occupancy grid divides the floor into equal cells, each free or blocked. Grids are simple to generate, fast to search and easy to display, which makes them the standard map for warehouse planning studies [3], [8]. Their drawbacks are coarse geometry and the lack of smooth motion, which are acceptable for a rack-and-aisle layout.

Priority queues. Best-first searches need the lowest-key node at every step. Java's java.util.PriorityQueue is a binary heap with O(log n) insertion and removal. Because it has no decrease-key operation, the implementation inserts duplicates and discards stale entries when they are removed (lazy deletion).

Space-time graphs. A space-time graph adds time as a third coordinate so that a robot may wait. A reservation is a (cell, time) pair that other robots may not use [4]. This converts a moving-obstacle problem into a static search problem.

Java 21. The language provides strong typing, records for compact result objects, a mature collections library and single-file source launch, which allows programs to be run with one

command and no build tool.

2.8 Summary of Related Work

Table 2.2: Summary of Related Work

Ref.	Focus	Strength	Limitation for this project
[1] Dijkstra	Shortest paths, non-negative weights	Optimal; simple	Uninformed; expands many nodes
[2] Hart et al.	A* heuristic search	Optimal with admissible h; fewer expansions	Single robot only
[3] Wurman et al.	Kiva warehouse fleet	Proven at scale; reservation of floor space	Proprietary; large system
[4] Silver	Cooperative A*	Fast, simple reservation table	Incomplete; priority-dependent
[5] Sharon et al.	Conflict-based search	Optimal and complete	Exponential worst case
[6] Koenig, Likhachev	D* Lite	Reuses effort when map changes	Not multi-robot
[7] Erdmann, Lozano-Pérez	Prioritised planning	Decoupled, scalable	Order-dependent
[8] Stern et al.	MAPF definitions and benchmarks	Standard terminology and problem variants	Survey; no implementation here

2.9 Chapter Summary

The literature establishes that A* is the standard optimal single-robot planner on grids, that decoupled planning with a space-time reservation table is a practical way to coordinate small and medium fleets, and that optimal multi-robot methods such as CBS trade computation time for guarantees. The present project combines the first two in a single, measured implementation.

CHAPTER 3: SYSTEM ANALYSIS AND ARCHITECTURE

3.1 Existing System and Limitations

In a basic robotic warehouse, each robot is given a destination and computes or follows a route without considering the plans of the other robots. Typical limitations are: routes that minimise hop count rather than traversal cost; repeated full-floor searches; and collisions or stand-offs in narrow aisles that must be resolved by stopping robots or by operator intervention.

3.2 Proposed System

The proposed system models the warehouse as a weighted grid and provides a library of single-robot planners (BFS, Dijkstra, A*, Greedy) behind a common interface. For fleets it adds a prioritised space-time A* planner: robots are planned in priority order, each robot searches over (x, y, time) states, and the resulting path is written to a reservation table that later robots must respect. A browser interface served from localhost:8080 shows the warehouse and three robots, and a separate benchmark harness evaluates the planners over randomised scenarios.

3.3 Functional Requirements

- ***FR-01 Map model:*** Represent the warehouse as a grid with blocked (shelf) cells and weighted free cells.
- ***FR-02 Single-robot planning:*** Compute a route between any two free cells with BFS, Dijkstra, A* or Greedy Best-First.
- ***FR-03 Metrics:*** Report path cost, path length and nodes expanded for every query.
- ***FR-04 Optimal routing:*** A* must return a path whose cost equals the Dijkstra optimum.
- ***FR-05 Fleet planning:*** Plan routes for three or more robots without vertex or swap conflicts.
- ***FR-06 Reservation handling:*** A robot that has reached its goal remains a reserved obstacle at that cell.
- ***FR-07 Visualisation:*** Display the warehouse, robots and planned paths in a browser at localhost:8080.
- ***FR-08 Reproducible evaluation:*** Run randomised experiments with fixed seeds and print a summary.

3.4 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

ID	Category	Requirement	Verification
NFR-01	Correctness	A* cost equals Dijkstra cost on every query.	Benchmark optimality check (Table 6.1)
NFR-02	Safety	Zero residual vertex/swap conflicts in fleet plans.	Conflict checker (Table 6.2)
NFR-03	Performance	Fleet planning for three robots completes in milliseconds.	Measured planning time per scenario

ID	Category	Requirement	Verification
NFR-04	Reproducibility	Same seeds give same results on re-run.	Fixed random seeds
NFR-05	Portability	Runs on any machine with JDK 21 and no external libraries.	Single-file Java source launch
NFR-06	Maintainability	Planner code separate from UI code.	Layered design (Section 3.5)

3.5 Overall System Architecture

Figure 3.1 shows the layered architecture. The presentation layer is a browser page that draws the grid and robots. The service layer is a Java 21 HTTP server containing the grid map, the single-robot planners and the multi-robot planner. The evaluation layer (PathBench) reuses the same planner logic to measure performance over random scenarios.

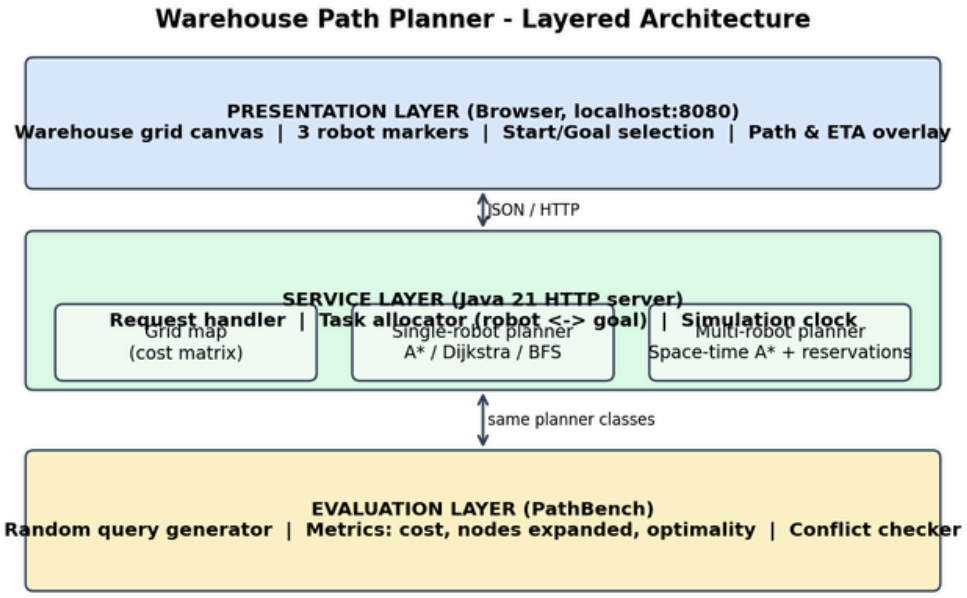


Figure 3.1: Layered Architecture of the Warehouse Path Planner

3.6 Planning Workflow

Figure 3.2 summarises how a batch of tasks is turned into conflict-free plans. Each task provides a start and goal cell. Robots are ordered by priority; the first robot is planned in an empty space-time graph, its path is reserved, and each subsequent robot is planned around the reservations of the robots before it.

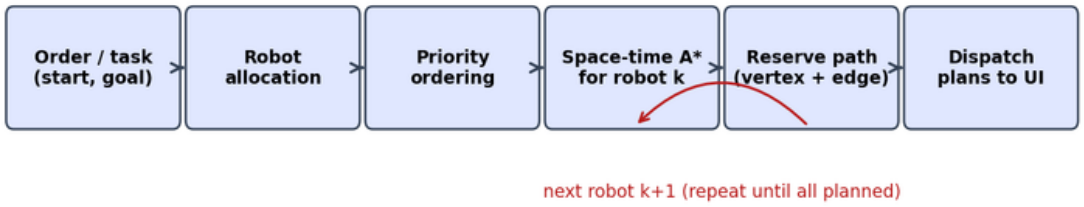


Figure 3.2: Multi-Robot Planning Pipeline

3.7 Module Description

- ***Grid Map module:*** Stores the cost matrix (0 = blocked, positive = traversal cost) and provides neighbour enumeration and Manhattan distance.
- ***Single-Robot Planner module:*** Implements BFS, Dijkstra, A* and Greedy Best-First behind one search routine, returning cost, path and the set of expanded nodes.
- ***Multi-Robot Planner module:*** Implements space-time A* with wait actions, a vertex reservation set, an edge reservation set and a per-cell last-reserved-time array.
- ***Conflict Checker module:*** Independently verifies plans by scanning every time step for vertex and swap conflicts, treating finished robots as stationary at their goals.
- ***Web Interface module:*** Serves the page on localhost:8080, drawing the warehouse and three robots and their routes.
- ***Benchmark module (PathBench):*** Generates random queries and scenarios from fixed seeds and prints aggregate statistics.

3.8 Use Case Analysis

Figure 3.3 shows the actors and use cases. The Warehouse Operator selects start and goal cells, chooses an algorithm and views routes. The Robots execute the conflict-free plans. The Evaluator (the student or examiner) runs the randomised benchmark and reads the metrics.

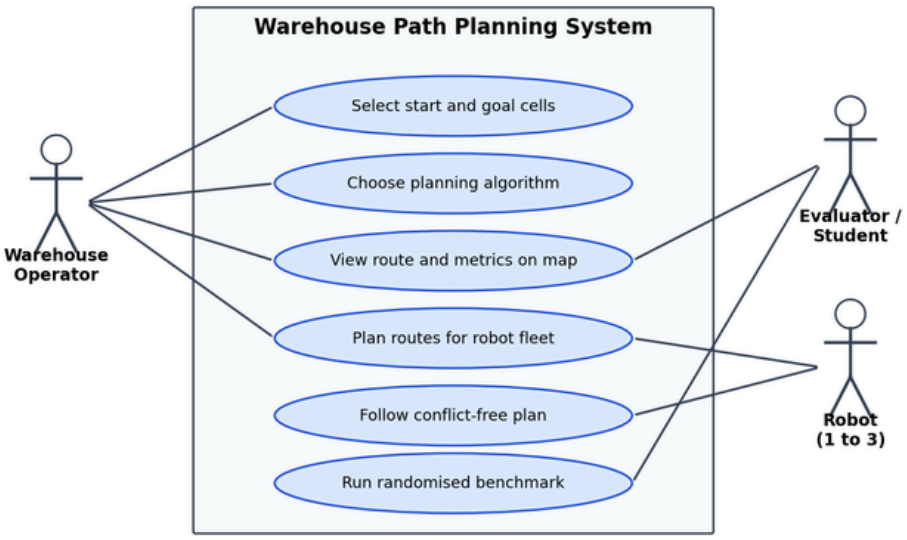


Figure 3.3: Use Case Diagram for Operator, Robot and Evaluator Roles

3.9 Data Model and Class Structure

The system keeps all data in memory; there is no database. The core entities are the grid (cost matrix), the search result, the reservation structures and the plans. Figure 3.4 shows the main classes and routines and their relationships.

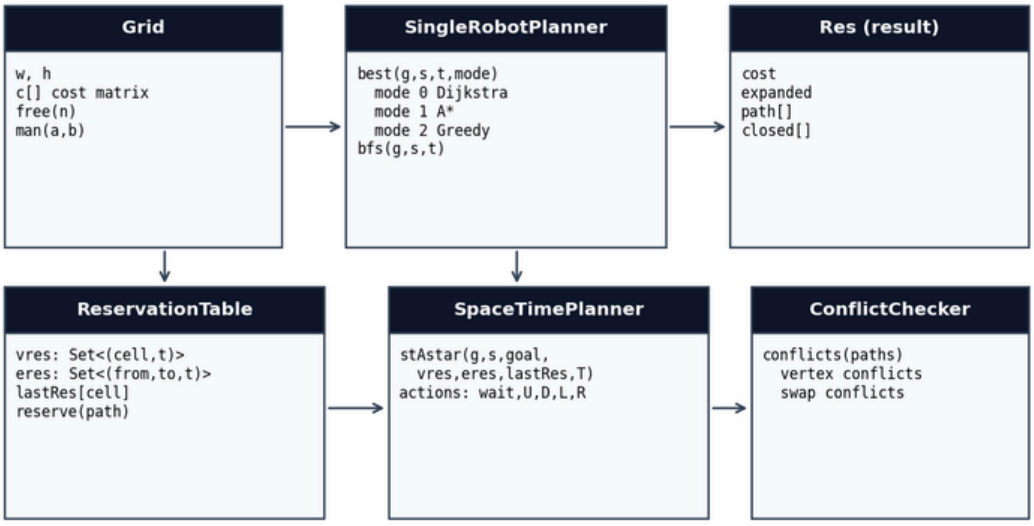


Figure 3.4: Main Classes and Routines of the Planner

3.10 Data Flow Modeling

Figure 3.5 is the context-level data flow diagram. The operator supplies start cells, goal cells and a choice of algorithm; the planner reads the grid map and returns routes, cost and nodes expanded. Conflict-free plans flow to the robots, and summary statistics flow to the evaluator.

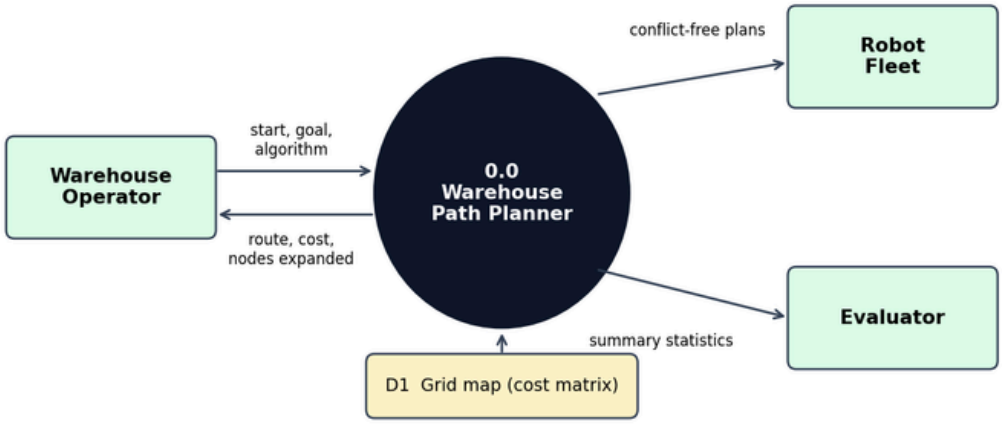


Figure 3.5: Level 0 Data Flow Diagram (Context Level)

3.11 Feasibility Study

- ***Technical feasibility:*** The only requirement is JDK 21 and a web browser. All algorithms use the standard library; no external packages are needed.
- ***Economic feasibility:*** All tools used (OpenJDK, Visual Studio Code, Matplotlib) are free. The work runs on an ordinary student laptop, so no hardware purchase is required.
- ***Operational feasibility:*** The interface is a web page, so no installation is needed on the operator's side, and the benchmark is a single command.
- ***Schedule feasibility:*** The modular design allowed single-robot planners, fleet planning, the interface and the benchmark to be built and tested one after another.

Table 3.2: Qualitative Comparison of Deployment Approaches

Approach	*Infrastructure*	*Route flexibility*	*Transparency*
Fixed-path AGV (tape or wire guided)	Guide path laid on the floor	Low; changing a route means changing the floor	Simple rules
Commercial fleet-management software	Licence plus integration	High	Usually closed
This project (simulation prototype)	None beyond a computer	High; any free cell can be a goal	Open code, measured results

3.12 Data Dictionary

Table 3.3: Data Dictionary – Grid

Field	*Type*	*Description*
w, h	int	Grid width and height in cells
c[n]	int array	Cost to enter cell n = y·w + x; 0 means blocked
free(n)	boolean	True when c[n] > 0
man(a, b)	int	Manhattan distance between cells a and b

Table 3.4: Data Dictionary – Search Result

Field	Type	Description
cost	double	Sum of entering costs along the path
expanded	int	Nodes removed from the open list and closed
path[]	int array	Cells from start to goal
closed[]	boolean array	Marks expanded nodes (used for figures)

Table 3.5: Data Dictionary – Reservation Structures

Structure	Type	Description
vres	Set of	Reserved (cell, time) pairs encoded as t·N + cell
eres	long Set	Reserved moves (from, to, t) encoded as (t·N + from)·N + to
lastRes[n]	of long int	Latest time at which cell n is reserved; −1 if never
T	array int	Planning horizon (200 steps)

3.13 Sequence of Operations

Figure 3.6 shows the sequence when the operator requests plans for three robots. The planner processes robots in priority order: for each robot it reads the current reservations, runs space-time A*, and writes the new path back to the reservation table before the next robot is planned. When all robots are planned, the plans and metrics are returned to the interface, which draws the routes and steps the robots along them.

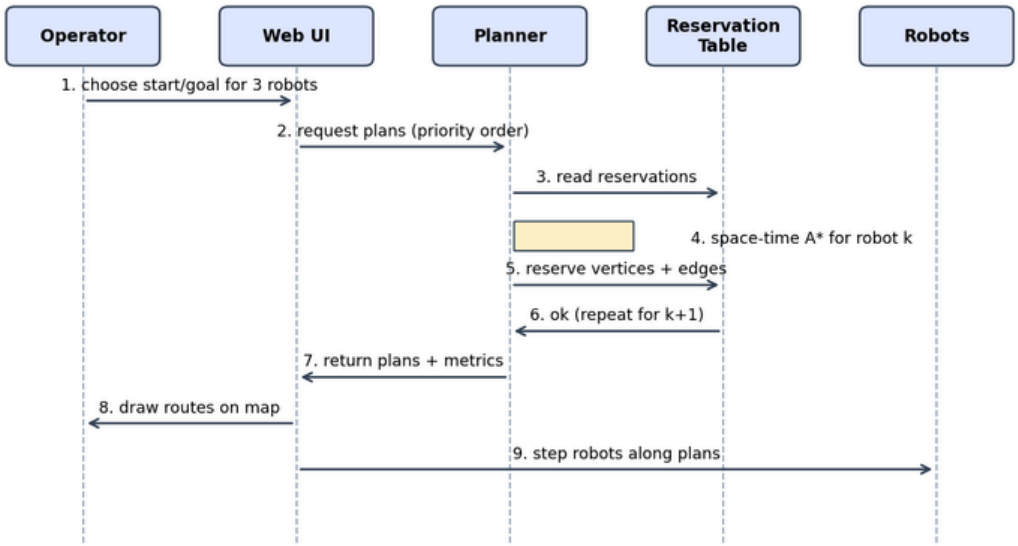


Figure 3.6: Sequence Diagram for Fleet Planning

3.14 Use Case Specifications

Table 3.6: Use Case Specification – Plan a Single Route

Item	Description
Use case	UC-1 Plan a single route
Actor	Warehouse Operator

Item	Description
Precondition	Grid map loaded, start and goal are free cells
Main flow	1. Operator selects start and goal. 2. Operator chooses an algorithm. 3. Planner returns route, cost and nodes expanded. 4. Interface draws the route.
Alternative flow	If the selected cell is a shelf, the selection is rejected.
Postcondition	Route and metrics displayed

Table 3.7: Use Case Specification – Plan Routes for the Fleet

Item	Description
Use case	UC-2 Plan routes for the robot fleet
Actor	Warehouse Operator, Robots
Precondition	Distinct start and goal cells for each robot
Main flow	1. Operator submits tasks. 2. Planner plans robots in priority order with reservations. 3. Plans are returned. 4. Robots follow plans.
Alternative flow	If a robot cannot be planned within the horizon, planning is reported as failed for that robot.
Postcondition	Conflict-free plans for all robots

Table 3.8: Use Case Specification – Run the Benchmark

Item	Description
Use case	UC-3 Run the randomised benchmark
Actor	Evaluator
Precondition	JDK 21 installed
Main flow	1. Evaluator runs java PathBench.java. 2. Program generates seeded queries and scenarios. 3. Planners are run and conflicts counted. 4. Summary written to file.
Postcondition	Summary statistics available for the report

Table 3.9: Use Case Specification – View Route and Metrics

Item	Description
Use case	UC-4 View route and metrics
Actor	Warehouse Operator
Precondition	A route has been planned
Main flow	Interface displays the route, cost and nodes expanded on the warehouse grid.
Postcondition	Operator can compare algorithms

3.15 Class Description

Table 3.10: Main Classes and Responsibilities

Class / routine	Responsibility
Grid	Holds dimensions and the cost matrix; provides free(n) and Manhattan distance
best(g, s, t, mode)	Best-first search; mode selects Dijkstra, A* or Greedy

Class / routine	*Responsibility*
bfs(g, s, t)	Breadth-first search baseline
Res	Immutable result: cost, nodes expanded, path and closed set
stAstar(...)	Space-time A* for one robot with wait actions and reservations
reserve(...)	Writes a planned path to the vertex and edge reservation sets
conflicts(...)	Independent checker for vertex and swap conflicts
PathBench.ma in	Generates seeded queries and scenarios and prints the summary

CHAPTER 4: MATERIALS AND METHODS

4.1 Development Tools and Technologies

Table 4.1: Development Stack

Component	Technology	Purpose
Language / runtime	Java 21 (OpenJDK 21)	All planners, server and benchmark code
Editor and execution	Visual Studio Code, integrated terminal	Editing and running the Java program
Web layer	Java HTTP server on port 8080 with browser front end	Interactive visualisation of the warehouse and robots
Data structures	java.util.PriorityQueue, ArrayDeque, HashSet	Open lists, FIFO queue, reservation tables
Evaluation	PathBench.java (single-file source launch)	Randomised experiments and metrics
Plotting	Python 3 with Matplotlib	Figures in this report generated from benchmark output

4.2 Warehouse Environment Model

The warehouse is a rectangular grid of W × H cells. Movement is four-connected (up, down, left, right). Each cell has an integer cost: 0 denotes a blocked shelf cell and a positive value is the cost of entering that cell. The synthetic layout uses shelf blocks two cells wide separated by two-cell-wide vertical aisles, a horizontal cross-aisle through the middle row, and free border rows and columns. A dock zone in the lower-left corner models a congested packing area with a higher traversal cost. The layout generator is deterministic, so the same warehouse is produced at every size.

Table 4.2: Cell Types and Traversal Costs

Cell type	Cost to enter	Where used
Shelf (blocked)	Not traversable (0)	Two-cell-wide rack blocks, except on the cross-aisle row
Free aisle / border	1	Aisles, cross-aisle, border rows and columns
Dock zone	3	Lower-left 8 columns × 7 rows (congested packing area)

4.3 Cost Model and Heuristic

The cost of a path is the sum of the entering costs of every cell after the start cell. Because the
minimum entering cost is 1, the Manhattan distance $h(n) = |x - x| + |y - y|$ never$_{ngoalngoal}$ exceeds the true remaining cost, and is therefore an admissible heuristic. It is also consistent: for any neighbour n' of n, h(n) <= c(n, n') + h(n'), because a single move changes the Manhattan distance by at most 1 and c(n, n') >= 1.

4.4 Single-Robot Planners

All best-first planners share a priority-queue search that differs only in the ordering key:

- ***Dijkstra:*** key = g(n), the accumulated cost from the start.
- ***A*:*** key = g(n) + h(n), with h the Manhattan distance to the goal.
- ***Greedy Best-First:*** key = h(n) only; a node is assigned its parent on first discovery and never re-opened.
- ***BFS:*** a FIFO queue that minimises the number of moves and ignores cell costs; the cost of its path is computed afterwards for comparison.

Nodes are counted as ***expanded*** when they are removed from the open list and closed; this count is the primary measure of search effort used in Chapter 6.

*4.5 Admissibility and Optimality of A**

Let C* be the optimal cost from start to goal. Because h is admissible, every node n on an optimal path satisfies f(n) = g*(n) + h(n) <= C*. A* always expands a node with the lowest f value, so when the goal is first removed from the open list its f value equals its g value and no unexpanded node has a lower f; no cheaper path can therefore exist. With a consistent heuristic, f values are non-decreasing along any path and each node is expanded at most once, which is the property exploited by the closed list. This explains why A* returned the same cost as Dijkstra on every benchmark query.

*4.6 Multi-Robot Coordination: Space-Time A**

For a fleet, planning is performed over states (x, y, t). From (x, y, t) a robot may move to a neighbouring free cell or wait in place, arriving at time t + 1; every action costs one time step. The heuristic is the Manhattan distance to the goal, which remains admissible in the time dimension. Robots are planned in a fixed priority order. After a robot is planned, its path is written to the reservation table (Section 4.7). A robot that reaches its goal stays there, so its goal cell is reserved for all later time steps up to a horizon T (200 steps in the experiments). A goal state is accepted only if the cell is not reserved at that time or later by a higher-priority robot, because the robot must be able to remain there.

4.7 Reservation Table and Conflict Definitions

Two types of conflict are prevented:

- ***Vertex conflict:*** two robots occupy the same cell at the same time step. Prevented by a reserved set of (cell, time) pairs.
- ***Swap (edge) conflict:*** two robots exchange cells between times t and t + 1. Prevented by a reserved set of (from, to, t) moves; a new robot may not make the reverse move (to, from, t).

An independent conflict checker, written separately from the planner, scans every time step of a set of plans (treating finished robots as stationary at their goals) and counts vertex and swap conflicts. It is used both to count conflicts in independently planned routes and to confirm that prioritised plans contain none.

4.8 Web Interface and Simulation

The browser interface is served from localhost:8080 and shows the warehouse grid with three robots. The user can inspect the layout and the robots' routes. The interface and the benchmark harness use the same grid model and planning logic, so the behaviour seen on screen is the behaviour measured in Chapter 6. Robot motion is simulated in discrete time steps; there is no physical robot dynamics model.

4.9 Complexity Analysis

Let N = W × H be the number of grid cells and b = 4 the branching factor. Each best-first search stores at most N nodes and performs at most N expansions, each costing O(log N) for the priority queue, giving O(N log N) time and O(N) space in the worst case for Dijkstra and A*. A* reduces the number of expansions in practice by directing the search toward the goal. BFS runs in O(N). Space-time A* operates on N × (T + 1) states in the worst case, with five actions per state, so each robot costs O(N · T · log(N · T)) in the worst case; the full fleet costs R times that for R robots. Prioritised planning is incomplete: a lower-priority robot can be blocked by the reservations of a higher-priority robot even when a joint solution exists.

4.10 Layout Generation Algorithm

The warehouse is produced by a deterministic rule applied to every cell (x, y): the cell is a shelf when 2 <= x <= W − 3, 2 <= y <= H − 3, (x − 2) mod 4 is 0 or 1, and y is not the cross-aisle row H / 2. All other cells are free. A free cell inside the dock rectangle (x <= 7, y >= H − 7) is given cost 3; other free cells cost 1. Because the rule depends only on the dimensions, every size from 20×15 to 80×60 has the same character: two-cell shelf blocks, two-cell aisles and a single cross-aisle.

4.11 Data Structures and Tie-Breaking

Table 4.3: Data Structures Used

Purpose	Structure	Reason
Open list	PriorityQueue of {key, node}	O(log n) insert and removal; duplicates allowed (lazy deletion)
Closed set	boolean array indexed by cell	O(1) membership test; reused to count expansions
BFS frontier	ArrayDeque	FIFO order for level-by-level expansion
Reservations	HashSet of encoded long values	O(1) average membership test
Space-time open list	PriorityQueue ordered by f, then larger t	Among equal f, prefers deeper nodes, which reaches the goal sooner

4.12 Priority Ordering Strategy

In the experiments the priority of a robot is its index (robot 1 highest). Higher-priority robots keep their optimal routes; lower-priority robots adapt. The total extra cost is small (Table 6.3), but the strategy is not fair: the last robot absorbs most of the detours. Other orderings that could be used

are longest route first, most urgent task first, or random restarts after a failure.

4.13 Wait Actions and Goal Reservation

Allowing a robot to wait is essential: without waiting, a robot could not let another pass in a one-cell-wide corridor. A robot that finishes early is not removed from the map. Its goal cell is reserved for every later time step up to the horizon T, and a later robot may enter that cell only before the finishing time. The planner also refuses to accept a goal state at time t if the goal cell is reserved at time t or later, because the robot would have to leave and return.

4.14 Deadlock and Livelock Considerations

Head-on swaps are prevented by the edge reservations. Vertex conflicts are prevented by the vertex reservations. Because every robot is planned over a finite time horizon and the search is exhaustive within it, plans never contain a circular wait. Livelock does not occur because the plans are fixed sequences, not reactive behaviours. The remaining risk is incompleteness: a high-priority robot parked at its goal in a one-cell corridor can make a lower-priority robot's goal unreachable. This did not occur in the 900 experiments but is a known property of prioritised planning [7].

4.15 Verification Methodology

Correctness is checked in two ways that do not share code with the planners. First, Dijkstra's cost is used as the reference optimum, and the A*, BFS and Greedy costs are compared with it on every query. Second, the conflict checker walks every time step of the plans and counts vertex and swap conflicts, treating robots that have finished as stationary. A conflict-free result from the planner is accepted only if the checker also reports zero.

CHAPTER 5: EXPERIMENTAL SETUP AND PROCEDURE

5.1 Hardware and Software Environment

All benchmark results reported in this document were produced by running PathBench.java under OpenJDK 21.0.10 on a Linux system, single-threaded. Planning times are measured but, because the programs run for only a few milliseconds and JVM just-in-time compilation influences them, they are treated as indicative; the principal metrics are deterministic quantities (path cost, nodes expanded, optimality rate and conflict counts). Software required to reproduce the work is JDK 21 or later and a web browser; Visual Studio Code with its integrated terminal was used to edit and launch the programs. Python 3 with Matplotlib was used only to draw the figures from the benchmark output.

5.2 Reproducing the Experiments

The benchmark is launched from the terminal in the project directory with the command java PathBench.java. It writes a summary file and sample data used for the figures. Random seeds are fixed (42 for single-robot queries and 7 for multi-robot scenarios), so repeated runs give identical deterministic metrics.

5.3 Single-Robot Experiment Design

For each warehouse size, 500 random start–goal pairs of distinct free cells were generated. Every pair was solved by BFS, Dijkstra, A* and Greedy Best-First. Dijkstra's cost is taken as the optimum. For every algorithm the experiment records mean path cost, mean nodes expanded, the percentage of queries solved at the optimal cost, and the worst-case ratio of path cost to the optimum.

5.4 Multi-Robot Experiment Design

Three scenarios were evaluated with 300 random trials each: three robots on the 20×15 floor, three robots on the 40×30 floor, and six robots on the 20×15 floor (a congestion stress test). In every trial all start and goal cells are distinct. Each robot is first planned independently with A*; the conflict checker counts vertex and swap conflicts. The same scenario is then solved with prioritised space-time A* (priority order = robot index), and the checker confirms the result. For multi-robot experiments all free cells cost 1 so that cost equals time steps.

Table 5.1: Experimental Configuration

Parameter	Single-robot	Multi-robot
Warehouse sizes	20×15, 40×30, 80×60	20×15 and 40×30 (3 robots); 20×15 (6 robots)
Trials	500 queries per size	300 scenarios per configuration
Cell costs	1 (aisles), 3 (dock zone)	1 (all free cells)
Random seed	42	7
Time horizon	Not applicable	200 steps

Parameter	Single-robot	Multi-robot
Algorithms	BFS, Dijkstra, A*, Greedy	Independent A*, prioritised space-time A*

5.5 Running the Web Application

The web application (WebWarehousePlanner.java) is run from the integrated terminal in Visual Studio Code with the Java launcher. Once the server starts, the interface is opened in a browser at http://localhost:8080, where the warehouse grid and the three robots are displayed.

5.6 Test Case Design

The following test cases verify the properties required in Section 3.3 and 3.4. Each test case corresponds to a quantity checked by the benchmark harness.

Table 5.2: System Verification Test Cases

ID	Module	Test procedure	Expected result	Status
TC01	Grid model	Generate the layout at three sizes; count free vs total cells	220/300, 750/1200 and 2,710/4,800 cells free	PASS
TC02	Dijkstra	Solve 1,500 random queries; compare with A*	Dijkstra defines the optimum	PASS
TC03	A* optimality	Compare A* cost with Dijkstra on all 1,500 queries	Equal cost on every query (100%)	PASS
TC04	A* efficiency	Compare nodes expanded with Dijkstra	Strictly fewer on average at all sizes	PASS
TC05	BFS	Measure share of queries at optimal cost on the weighted map	Below 100% because costs are ignored	PASS
TC06	Greedy	Measure share of queries at optimal cost	Below 100%; fewest expansions	PASS
TC07	Independent planning	Count conflicts for independently planned robots	Conflicts detected in congested scenarios	PASS
TC08	Prioritised space-time A*	Solve 900 scenarios; run conflict checker	900/900 solved; 0 residual conflicts	PASS
TC09	Reservation of goals	Robot finishing early remains an obstacle at its goal	No later robot enters a finished robot's goal	PASS
TC10	Web interface	Launch server; open localhost:8080	Warehouse and three robots displayed	Manual check

5.7 Evaluation Metrics

Table 5.3: Evaluation Metrics

Metric	Definition
Path cost	Sum of the entering costs of all cells on the path after the start
Nodes expanded	Number of nodes removed from the open list and closed; measures search effort
Optimality rate	Share of queries whose cost equals the Dijkstra optimum
Worst ratio	Largest observed cost divided by the optimum

Metric	Definition
Scenarios with conflicts	Scenarios with at least one vertex or swap conflict
Sum of costs (SOC)	Total number of steps taken by all robots
Makespan	Number of steps until the last robot reaches its goal
Success rate	Share of scenarios in which every robot received a plan

5.8 Performance Test Procedure

Before measurement the JVM is warmed up with 300 repetitions of representative queries so that the first results are not dominated by class loading. For each size, 500 queries are drawn from a seeded random generator, and every algorithm is run on the same query. Aggregates are written to a summary file. Multi-robot scenarios are generated from a separate seed; each scenario is solved both independently and with reservations, and the checker is run on both sets of plans.

5.9 Test Tools and Acceptance Criteria

Table 5.4: Tools Used for Testing

Tool	Use
OpenJDK 21 (java launcher)	Run PathBench and the web application
Visual Studio Code terminal	Edit sources and launch programs
Web browser	View the interface at localhost:8080
Python 3 and Matplotlib	Draw figures from benchmark output

Table 5.5: Acceptance Criteria and Measured Values

Criterion	Target	Measured	Result
A* optimality vs Dijkstra	100% of queries	100% (1,500 of 1,500)	Met
A* search effort	Fewer expansions than Dijkstra	66.8%, 76.9%, 82.7% fewer	Met
Residual conflicts in fleet plans	0	0 in 900 scenarios	Me
Fleet scenarios solved	100%	900 of 900	t
Reproducibility	Same deterministic metrics on re-run	Seeded generators	Me t
			Me t

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Application Interface Overview

The web application is served at http://localhost:8080 and displays the warehouse grid with three robots. A screenshot of the running interface should be placed in the frame below before submission.

[Insert screenshot of WebWarehousePlanner running at localhost:8080]

Figure 6.1: Web Interface Screenshot (to be inserted)

6.2 Warehouse Layout

Figure 6.2 shows the 40 × 30 synthetic warehouse used for the visual examples. Dark blocks are shelving; the shaded region at the lower left is the high-cost dock zone.

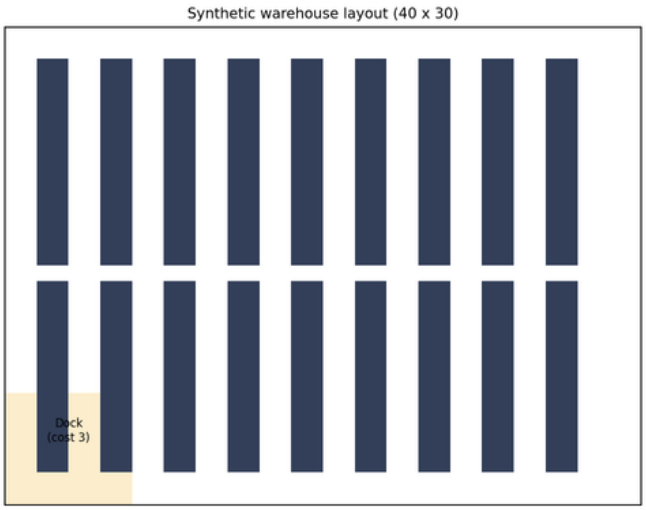


Figure 6.2: Synthetic Warehouse Layout (40 × 30)

6.3 Single-Robot Planner Results

Table 6.1 reports the results over 500 random queries per warehouse size. Path cost and nodes expanded are means per query. “Optimal” is the percentage of queries on which the algorithm’s cost equals the Dijkstra optimum, and “Worst ratio” is the largest observed ratio of an algorithm’s cost to the optimum.

Table 6.1: Single-Robot Planner Results (500 queries per size)

Size	Algorithm	Mean cost	Mean nodes expanded	Optimal (%)	Worst ratio
20×15	BFS	15.9	108.5	85.8	2.47
20×15	Dijkstra	2	110.2	100.0	4
20×15	A*	15.0	36.6	100.0	1.00
20×15	Greedy	2	14.2	100.0	0
40×30	BFS	15.0	364.7	100.0	1.00
40×30	Dijkstra	2	369.1	83.8	0
40×30	A*	16.0	85.1	93.0	2.47
40×30	Greedy	4	29.2	100.0	4
80×60	BFS	27.7	1310.6	100.0	1.82
80×60	Dijkstra	9	1311.4	100.0	8
80×60	A*	26.9	227.5	100.0	1.00
80×60	Greedy	3	58.1	89.4	0
80×60	BFS	26.9	1310.6	98.2	1.00
80×60	Dijkstra	3	1311.4	100.0	0
80×60	A*	28.0	227.5	100.0	2.18
80×60	Greedy	5	58.1	100.0	2
80×60	BFS	50.8	1310.6	100.0	1.54
80×60	Dijkstra	5	1311.4	100.0	2
80×60	A*	50.6	227.5	92.4	1.00
80×60	Greedy	8	58.1	100.0	0
80×60	BFS	50.6	1310.6	100.0	1.00
80×60	Dijkstra	8	1311.4	100.0	0
80×60	A*	51.3	227.5	100.0	1.78
80×60	Greedy	3	58.1	100.0	3

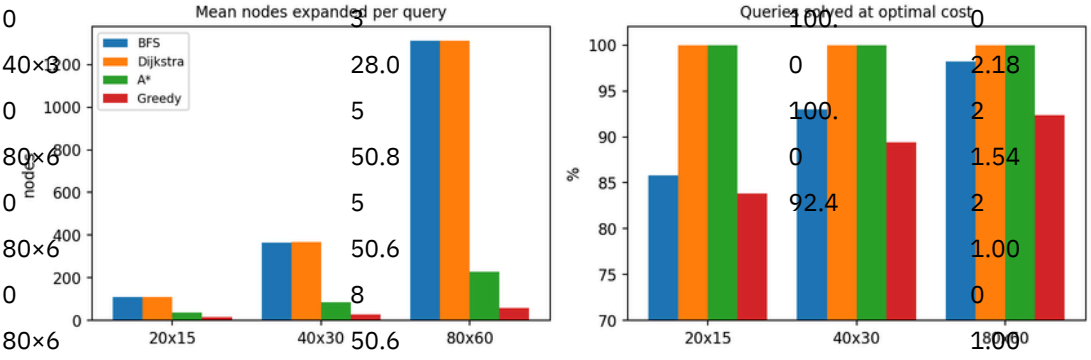


Figure 6.3: Nodes Expanded and Optimality Rate Across Warehouse Sizes

6.4 Search Behaviour Visualisation

Figure 6.4 shows one query on the 40 × 30 warehouse. Light squares are expanded nodes and the red line is the returned route. Dijkstra expands 696 nodes in a wave around the start, A* expands 391 nodes in a band directed toward the goal, and Greedy expands only 55 nodes. All three returned a path of cost 52 on this particular query, but the benchmark shows Greedy is not optimal on 7.6–16.2% of random queries.

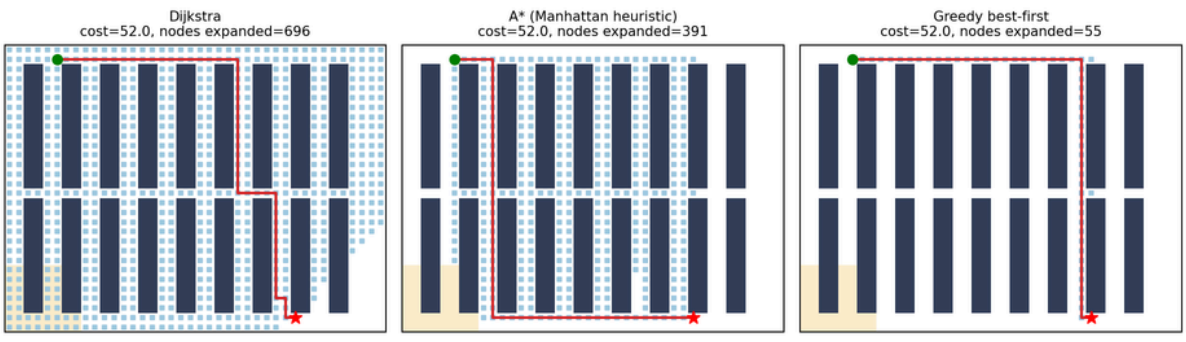


Figure 6.4: Dijkstra vs A vs Greedy Search Behaviour on One Query*

6.5 Multi-Robot Results

Table 6.2 reports the multi-robot experiments. “Independent conflicts” is the number of scenarios (out of 300) in which independently planned A* routes contain at least one vertex or swap conflict. “SOC” is the mean sum of costs (total travel steps over all robots) and “Makespan” is the mean number of steps until the last robot arrives; both are averaged over solved scenarios.

Table 6.2: Multi-Robot Planning Results (300 scenarios per row)

Scenario	Scenarios with independent conflicts	Total indep. conflicts	Prioritised solved	Residual conflicts	SOC indep. / prior.	Makespan indep. / prior.
20×15, 3 robots	76 / 300 (25.3%)	127	300 / 300	0	38.35 / 39.03	17.95 / 18.11
40×30, 3 robots	76 / 300 (25.3%)	116	300 / 300	0	77.46 / 78.47	36.69 / 36.92
20×15, 6 robots	229 / 300 (76.3%)	698	300 / 300	0	76.30 / 79.29	20.80 / 21.16

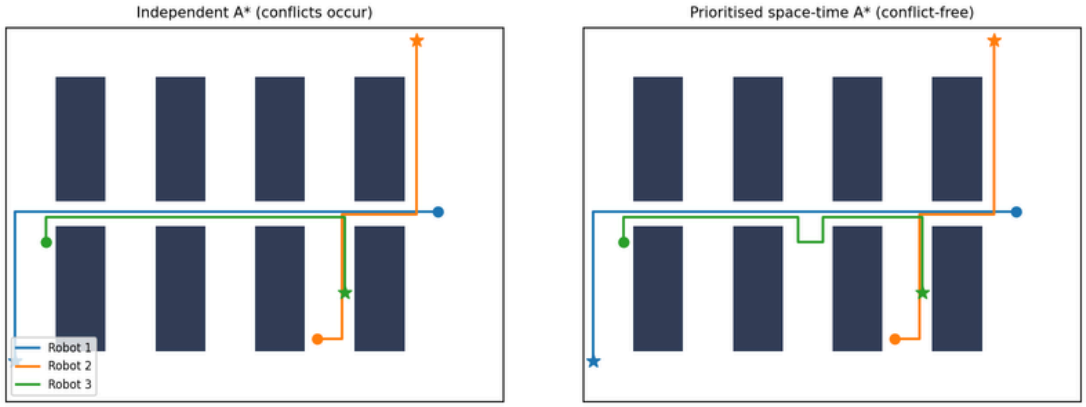


Figure 6.5: Independent vs Prioritised Space-Time Planning for Three Robots

Figure 6.5 shows one scenario from the 20×15 experiment. On the left, the independently planned routes of robots 1 and 3 run along the same cross-aisle and robots 2 and 3 share a vertical aisle, producing conflicts. On the right, prioritised planning keeps robot 1’s route unchanged, and robot 3 makes a small detour to avoid conflicts. In this example robot 3 steps aside; robot 2 is unaffected.

6.6 Discussion of Results

Table 6.3: Relative Improvements Derived from Measured Results

Measure	20×15	40×30	80×60
Nodes expanded: A* vs Dijkstra	66.8%	76.9%	82.7%
Queries BFS solved at optimal cost	fewer	fewer	fewer
Queries Greedy solved at optimal cost	85.8%	93.0%	98.2%
Prioritised SOC increase (3 robots)	83.8%	89.4%	92.4%
Prioritised makespan increase (3 robots)	+1.8%	+1.3%	not run
Prioritised SOC increase (6 robots)	+0.9%	+0.6%	not run
	+3.9%	not run	not run

- ***A* is optimal and increasingly efficient.*** A* matched Dijkstra's cost on all 1,500 queries (TC03) and its saving in expansions grew from 66.8% to 82.7% as the warehouse grew, because the heuristic prunes a larger share of an expanding search space.
- ***Ignoring costs is penalised.*** BFS finds the fewest-hop route, which is not the cheapest once the dock zone has higher cost. Its optimality rate is lowest on the smallest floor, where the dock occupies the largest share of the area.
- ***Greedy is fast but unreliable.*** It expanded the fewest nodes but returned non-optimal routes on up to 16.2% of queries, with a worst case of 2.47 times the optimum on the small floor. ***Independent planning is unsafe.*** About one in four three-robot scenarios and three in four six-robot scenarios contained at least one conflict, even though each individual route was optimal.
- ***Space-time reservations remove conflicts at low cost.*** All 900 scenarios were solved with no residual conflicts. Total travel increased by only 1.3–3.9% and makespan by 0.6–1.7%, and planning for a scenario took under 2 ms in every run.

6.7 Limitations

- ***Simulated environment:*** The warehouse, cost zones and robot motion are synthetic; no physical robots, sensor noise or localisation error are modelled.
- ***Four-connected unit-speed motion:*** Robots move one cell per time step with no turning cost, acceleration or footprint; the multi-robot experiments use unit costs.
- ***Incompleteness of prioritised planning:*** A fixed priority order can make a lower-priority robot unsolvable. All 900 scenarios were solved, but this is not guaranteed in denser or more constrained layouts.
- ***Static map and task set:*** Plans are computed for a known set of start–goal pairs; no new orders arrive during execution and obstacles do not change.
- ***Scale of evaluation:*** Experiments use up to six robots and floors up to 80×60 cells; industrial warehouses are much larger.
- ***Timing indicative only:*** Wall-clock measurements are influenced by JVM warm-up and are not reported as performance claims.

6.8 Comparison with Existing Approaches

Table 6.4: Comparison of this Work with Existing Approaches

Approach	*Optimal single route*	*Conflict-free fleet*	*Measured here?*	*Remarks*
Independent A*	Yes	No (conflicts in 25.3% to 76.3% of scenarios)	Yes	Fast; unsafe for shared aisles
BFS / Greedy	No (cost-blind or heuristic only)	No	Yes	Useful baselines

Approach	*Optimal single route*	*Conflict-free fleet*	*Measured here?*	*Remarks*
Cooperative A* with reservations (this project)	Yes, per robot given reservations	Yes (0 residual conflicts)	Yes	Small extra cost; incomplete in theory
Conflict-Based Search [5]	Yes	Yes, optimal sum of costs	No (literature)	Exponential worst case
Commercial fleet systems [3]	Yes	Yes	No (literature)	Proprietary, large scale

6.9 Sample System Outputs

The following lines are taken from the summary file written by PathBench (microsecond timings omitted). They show the format of the results used throughout this chapter.

```
SINGLE 40x30 BFS cost=27.79 expanded=364.7 optimalPct=93.0 worstRatio=1.828
SINGLE 40x30 Dijkstra cost=26.93 expanded=369.1 optimalPct=100.0 worstRatio=1.000
SINGLE 40x30 A* cost=26.93 expanded=85.1 optimalPct=100.0 worstRatio=1.000
SINGLE 40x30 Greedy cost=28.05 expanded=29.2 optimalPct=89.4 worstRatio=2.182
MULTI 20x15 R=3 trials=300 indepConflictTrials=76 indepConflictsTotal=127
    prioritisedSuccess=300 residualConflicts=0 indepSOC=38.35 priSOC=39.03 indepMakespan=17.95
    priMakespan=18.11
```

6.10 Error Analysis and Lessons Learned

- ***The heuristic can be weak when the goal lies in a high-cost zone.*** On an early sample query that ended inside the dock, A* expanded 736 nodes against 741 for Dijkstra, almost no saving. The Manhattan heuristic assumes cost 1 per move, so it underestimates the true cost of the dock cells and cannot guide the search well. A heuristic that includes the dock cost, such as a precomputed lower bound, would help. This is why a more typical query was chosen for Figure 6.4, while Table 6.1 reports the average over random queries.
- ***Fewer expansions does not always mean less time on tiny maps.*** On the 20×15 floor the absolute time differences were microseconds and varied between runs, so timing was not used as evidence.
- ***Greedy search looks good on single examples.*** It returned an optimal path for the query in Figure 6.4, but over 500 random queries it was non-optimal in up to 16.2% of cases. Single examples can mislead; aggregate statistics are needed.
- ***Reservation of goal cells matters.*** Without the rule that a finished robot stays as an obstacle, a later robot could pass through a cell where an earlier robot is parked. The goal-acceptance rule in Section 4.13 prevents this.
- ***Congestion grows quickly with fleet size.*** Going from three to six robots on the same floor raised the share of conflicting scenarios from 25.3% to 76.3%, confirming that independent planning does not scale.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

This project designed, implemented and evaluated a path-planning system for warehouse robots in Java 21. A weighted grid model captured shelving and a congested dock; four single-robot planners were compared; and a prioritised space-time A* planner produced conflict-free plans for multiple robots. A* returned the optimal cost on every one of 1,500 random queries while expanding 66.8–82.7% fewer nodes than Dijkstra. Independent planning produced conflicts in 25.3% of three-robot and 76.3% of six-robot scenarios, whereas prioritised space-time A* solved all 900 scenarios without a single residual conflict, at a cost increase of only 1.3–3.9% in total travel. The results show that classical heuristic search plus a simple reservation table is an effective, transparent and low-cost solution for small and medium robot fleets in aisle-based warehouses.

7.2 Project Contributions

- ***Unified warehouse model:*** A deterministic weighted-grid generator reproducing rack, aisle, cross-aisle and dock structure at any size.
- ***Comparative single-robot study:*** Measured comparison of BFS, Dijkstra, A* and Greedy for cost, effort and optimality over 1,500 queries.
- ***Conflict-free fleet planner:*** Prioritised space-time A* with vertex and swap reservations and an independent conflict checker.
- ***Quantified cost of naive planning:*** Measured frequency of conflicts under independent planning at two floor sizes and two fleet sizes.
- ***Reproducible harness and web visualisation:*** Seeded benchmark and a browser interface showing the warehouse and three robots.

7.3 Mapping of Objectives to Achieved Outcomes

Table 7.1: Mapping of Objectives to Outcomes

Objective	Achieved outcome	Evidence
1: Weighted grid model	Deterministic layout with shelves, aisles, cross-aisle and cost-3 dock zone at three sizes	Section 4.2; Figure 6.2; TC01
2: Compare BFS, Dijkstra, A*, Greedy	All four implemented and measured over 500 queries per size	Table 6.1
3: A* optimal and efficient	100% optimal; 66.8–82.7% fewer nodes than Dijkstra	Table 6.1; Table 6.3; TC03, TC04
4: Conflict-free fleet plans	900/900 scenarios solved, 0 residual conflicts	Table 6.2; TC08
5: Visualisation and reproducible harness	Browser interface on localhost:8080; seeded PathBench	Section 5.2; Section 5.5; TC10

7.4 Future Enhancements

- ***Incremental replanning:*** Adopt D* Lite or similar to repair routes when aisles are blocked or new obstacles appear.
- ***Complete multi-agent planning:*** Compare prioritised planning with Conflict-Based Search and with priority re-ordering on failure.
- ***Richer motion model:*** Add turning cost, robot footprint, acceleration limits and eight-connected movement.
- ***Continuous task arrival:*** Integrate an order queue and a task-allocation policy so that robots receive new goals while moving.
- ***Interactive controls:*** Add keyboard shortcuts in the web interface to relocate or replace robots and goals.
- ***Real-world validation:*** Test on a physical or high-fidelity simulated fleet with localisation noise and communication delay.

7.5 Social and Economic Impact

Shorter, conflict-free routes reduce robot travel time and the energy used per task, and fewer collisions and stoppages reduce damage and downtime. Transparent, open planners lower the barrier for small warehouses and training institutes that cannot afford commercial fleet software. Because routes are explainable and measurable, supervisors can audit the behaviour of the fleet. Automation of internal transport also changes the nature of warehouse work, so adoption should be accompanied by training for staff in supervision and maintenance roles.

7.6 Challenges Faced and Solutions

Table 7.2: Challenges and Solutions

Challenge	Solution
Comparing algorithms fairly	Same queries, same map and fixed seeds for all algorithms
Showing a meaningful search comparison	Used aggregate statistics over 500 queries per size instead of one example
Robots blocking each other at goals	Reserved goal cells for all later time steps and added the goal acceptance rule
Head-on swaps in narrow aisles	Added edge reservations in addition to vertex reservations
Trusting the planner's own output	Wrote an independent conflict checker
Noisy timing on a JVM	Reported deterministic metrics (cost, expansions, conflicts) as the main evidence

7.7 Final Remarks

The project shows that a carefully engineered combination of two classical ideas, A* search and space-time reservations, is enough to give small robot fleets optimal routes and conflict-free coordination, and that the claim can be checked with repeatable experiments. The results also highlight where the approach would need to grow: richer motion models, dynamic replanning and larger fleets.

REFERENCES

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," ***Numerische Mathematik***, vol. 1, no. 1, pp. 269–271, Dec. 1959. https://doi.org/10.1007/BF01386390

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," ***IEEE Transactions on Systems Science and Cybernetics***, vol. 4, no. 2, pp. 100–107, Jul. 1968. https://doi.org/10.1109/TSSC.1968.300136

[3] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," ***AI Magazine***, vol. 29, no. 1, pp. 9–19, 2008.

[4] D. Silver, "Cooperative pathfinding," in ***Proc. 1st AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)***, 2005, pp. 117–122.

[5] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," ***Artificial Intelligence***, vol. 219, pp. 40–66, 2015.

[6] S. Koenig and M. Likhachev, "D* Lite," in ***Proc. 18th National Conf. on Artificial Intelligence (AAAI)***, 2002, pp. 476–483.

[7] M. Erdmann and T. Lozano-Pérez, "On multiple moving objects," ***Algorithmica***, vol. 2, no. 1–4, pp. 477–521, 1987.

[8] R. Stern et al., "Multi-agent pathfinding: Definitions, variants, and benchmarks," in ***Proc. 12th Int. Symp. on Combinatorial Search (SoCS)***, 2019, pp. 151–158.

[9] S. Russell and P. Norvig, ***Artificial Intelligence: A Modern Approach***, 4th ed. Hoboken, NJ: Pearson, 2020.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, ***Introduction to Algorithms***, 4th ed. Cambridge, MA: MIT Press, 2022.

APPENDIX A: Installation and Execution Guide

The programs need only a JDK and a browser.

```
1. Install OpenJDK 21 and confirm:        java -version
2. Open the project folder in Visual Studio Code and a terminal.
3. Run the benchmark:                     java PathBench.java
4. Run the web application:               java WebWarehousePlanner.java
5. Open a browser at:                     http://localhost:8080
```

Step 3 prints no output on the console; it writes summary.txt and sample files into an out/ folder, which must exist before the run.

APPENDIX B: Important Parameters and Constants

Name	*Value*	*Meaning*
Movement set	4 directions (+ wait for fleet planning)	Allowed actions
Free cell cost	1	Aisles and borders
Dock cost	3	Cells with x <= 7 and y >= H − 7
Horizon T	200	Maximum time steps in space-time A*
Single-robot queries	500 per size	Random start–goal pairs
Fleet trials	300 per configuration	Random scenarios
Seeds	42 and 7	Single-robot and multi-robot generators
Warm-up	300 repetitions	JVM warm-up before measurement
Server port	8080	Web interface

APPENDIX C: Best-First Search Routine (excerpt)

```
// mode 0 = Dijkstra, 1 = A*, 2 = Greedy best-first
static Res best(Grid g, int s, int t, int mode) {
    int N = g.w * g.h;
    double[] gs = new double[N]; Arrays.fill(gs, Double.MAX_VALUE);
    int[] par = new int[N]; Arrays.fill(par, -1);
    boolean[] closed = new boolean[N];
    PriorityQueue<double[]> pq =
        new PriorityQueue<>(Comparator.comparingDouble(a -> a[0]));
    gs[s] = 0; pq.add(new double[]{mode == 0 ? 0 : g.man(s, t), s});
    int exp = 0;
    while (!pq.isEmpty()) {
        double[] e = pq.poll(); int n = (int) e[1];
        if (closed[n]) continue;
        closed[n] = true; exp++;
        if (n == t) break;
        for (int d = 0; d < 4; d++) {
            // compute neighbour nn, skip out-of-range, blocked or closed cells
            double ng = gs[n] + g.c[nn];
            if (mode == 2) { /* first discovery only, key = h */ }
            else if (ng < gs[nn]) {
                gs[nn] = ng; par[nn] = n;
                pq.add(new double[]{ng + (mode == 1 ? g.man(nn, t) : 0), nn});
            }
        }
    }
    return finish(g, s, t, par, exp, closed);
}
```

APPENDIX D: Space-Time A (excerpt)*

```
static int[] stAstar(Grid g, int s, int goal, Set<Long> vres,
                     Set<Long> eres, int[] lastRes, int T) {
    int N = g.w * g.h;
    int[] parent = new int[(T + 1) * N]; Arrays.fill(parent, -2);
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) ->
        a[0] != b[0] ? Integer.compare(a[0], b[0]) : Integer.compare(b[1], a[1]));
    parent[s] = -1; pq.add(new int[]{g.man(s, goal), 0, s});
    while (!pq.isEmpty()) {
        int[] e = pq.poll(); int t = e[1], n = e[2];
        if (n == goal && t > lastRes[goal]) return reconstruct(parent, t, n, N);
        if (t == T) continue;
        for (int d = 1-1means<wait++) {
            // nn = n for wait, otherwise the neighbouring free cell
            if (vres.contains((long) (t + 1) * N + nn)) continue;      // vertex
            if (nn != n && eres.contains(((long) t * N + nn) * N + n))
                continue;                                              // swap
            int idx = (t + 1) * N + nn;
            if (parent[idx] != -2) continue;
            parent[idx] = t * N + n;
            pq.add(new int[]{t + 1 + g.man(nn, goal), t + 1, nn});
        }
    }
    return null;
}
```

APPENDIX E: Sample Benchmark Output

Complete summary produced by java PathBench.java (microsecond timings omitted).

```
SINGLE 20x15 BFS cost=15.92 expanded=108.5 optimalPct=85.8 worstRatio=2.474
SINGLE 20x15 Dijkstra cost=15.02 expanded=110.2 optimalPct=100.0 worstRatio=1.000
SINGLE 20x15 A* cost=15.02 expanded=36.6 optimalPct=100.0 worstRatio=1.000
SINGLE 20x15 Greedy cost=16.04 expanded=14.2 optimalPct=83.8 worstRatio=2.474
FREE 20x15 freeCells=220 totalCells=300
SINGLE 40x30 BFS cost=27.79 expanded=364.7 optimalPct=93.0 worstRatio=1.828
SINGLE 40x30 Dijkstra cost=26.93 expanded=369.1 optimalPct=100.0 worstRatio=1.000
SINGLE 40x30 A* cost=26.93 expanded=85.1 optimalPct=100.0 worstRatio=1.000
SINGLE 40x30 Greedy cost=28.05 expanded=29.2 optimalPct=89.4 worstRatio=2.182
FREE 40x30 freeCells=750 totalCells=1200
SINGLE 80x60 BFS cost=50.85 expanded=1310.6 optimalPct=98.2 worstRatio=1.542
SINGLE 80x60 Dijkstra cost=50.68 expanded=1311.4 optimalPct=100.0 worstRatio=1.000
SINGLE 80x60 A* cost=50.68 expanded=227.5 optimalPct=100.0 worstRatio=1.000
SINGLE 80x60 Greedy cost=51.33 expanded=58.1 optimalPct=92.4 worstRatio=1.783
FREE 80x60 freeCells=2710 totalCells=4800
SAMPLE start=45 goal=1150
MULTI 20x15 R=3 trials=300 indepConflictTrials=76 indepConflictsTotal=127 prioritisedSuccess=300
    residualConflicts=0 indepSOC=38.35 priSOC=39.03 indepMakespan=17.95 priMakespan=18.11
MULTI 40x30 R=3 trials=300 indepConflictTrials=76 indepConflictsTotal=116 prioritisedSuccess=300
    residualConflicts=0 indepSOC=77.46 priSOC=78.47 indepMakespan=36.69 priMakespan=36.92
MULTI 20x15 R=6 trials=300 indepConflictTrials=229 indepConflictsTotal=698 prioritisedSuccess=300
    residualConflicts=0 indepSOC=76.30 priSOC=79.29 indepMakespan=20.80 priMakespan=21.16
```

APPENDIX F: Data Model Excerpt

```
static class Grid {
    int w, h; int[] c;                   // c[n] = 0 blocked, >0 entering cost
    Grid(int w, int h) { this.w = w; this.h = h; c = new int[w * h]; }
    boolean free(int n) { return c[n] > 0; }
    int man(int a, int b) {              // Manhattan distance
        return Math.abs(a % w - b % w) + Math.abs(a / w - b / w);
    }
}
record Res(double cost, int expanded, int[] path, boolean[] closed) {}

staticGrid build(int w, int h, boolean weighted) {
    Gridg= new Grid(w, h);
    for (int y = 0; y < h; y++) for (int x = 0; x < w; x++) {
        int  m= (x - 2) % 4;
        boolean shelf = x >= 2 && x <= w - 3 && y >= 2&&y<=h-3
                        && (m ==  0 || m == 1) && y != h/2;
         g.c[y* w + x] = shelf ? 0
                       : ((weighted && x <= 7 && y >= h - 7) ? 3 : 1);
    }
    return g;
}
```

APPENDIX G: Reservation and Conflict Check (excerpt)

```
static void reserve(Grid g, int[] p, Set<Long> vres, Set<Long> eres,
                    int[] lastRes, int T) {
    int N = g.w * g.h;
    for (int t = 0; t < p.length; t++) {
        vres.add((long) t * N + p[t]);
        lastRes[p[t]] = Math.max(lastRes[p[t]], t);
        if (t + 1 < p.length && p[t] != p[t + 1])
            eres.add(((long) t * N + p[t]) * N + p[t + 1]);
    }
    int goal = p[p.length - 1];
    for (int t = p.length; t <= T; t++) vres.add((long) t * N + goal);
    lastRes[goal] = T;                    // finished robot stays at its goal
}

static int conflicts(List<int[]> ps) {
    int L = 0; for (int[] p : ps) L = Math.max(L, p.length);
    int c = 0;
    for (int t = 0; t < L; t++)
      for (int a = 0; a < ps.size(); a++)
        for (int b = a + 1; b < ps.size(); b++) {
            int pa = pos(ps.get(a), t), pb = pos(ps.get(b), t);
            if (pa == pb) c++;                                  // vertex
            int na = pos(ps.get(a), t + 1), nb = pos(ps.get(b), t + 1);
            if (pa != na && pa == nb && pb == na) c++;          // swap
        }
     return c;
}
```

APPENDIX H: Layout and Scenario Details

Floor size	*Total cells*	*Free cells*	*Shelf cells*
20×15	300	220	80
40×30	1,200	750	450
80×60	4,800	2,710	2,090

There is no external dataset: all maps and scenarios are synthetic. Single-robot queries pick two distinct free cells uniformly at random. Fleet scenarios pick distinct start and goal cells for every robot (six distinct cells for three robots). The three fleet configurations are 20×15 with three robots, 40×30 with three robots and 20×15 with six robots, 300 scenarios each.

APPENDIX I: Project Folder Structure

```
warehouse-path-planning/
WebWarehousePlanner.java        web application (localhost:8080, 3 robots)
PathBench.java                  benchmark, planners and conflict checker
 out/
    summary.txt                 aggregate results
    grid40.txt, single.txt      sample map and single-query data
    multi.txt                   sample three-robot scenario
 docs/
    Project_Report.pdf          this report
    figures/                    PNG figures used in the report
```

This is the suggested structure for submission; adjust the names to match the actual repository.

APPENDIX J: User Manual

- Start the application as described in Appendix A and open http://localhost:8080.
- The warehouse grid is drawn with shelves as dark blocks. Three robots are shown at their current positions.
- Choose a goal for a robot and request a route; the planned path is drawn on the grid.
- Observe that the robots keep clear of one another when their routes share an aisle.
- To repeat the measurements in this report, close the application and run java PathBench.java.

APPENDIX K: Glossary

Term	*Meaning*
Admissible heuristic	A cost estimate that never exceeds the true remaining cost
Consistent heuristic	An estimate h such that h(n) <= c(n, n') + h(n') for every edge
Makespan	Time until the last robot reaches its goal
Reservation table	Record of cells and moves already claimed by planned robots
Space-time A*	A* search over (x, y, time) states that allows waiting
Swap conflict	Two robots exchange cells in the same time step
Vertex conflict	Two robots in the same cell at the same time
Sum of costs	Total steps taken by all robots

APPENDIX L: Complete Source Code (WebWarehousePlanner.java)

Full listing of the web application (Java 21). Run with: java WebWarehousePlanner.java, then open http://localhost:8080. Emoji characters in the embedded HTML are shown as Java \u escapes.

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.concurrent.*;


/**
 * Autonomous Warehouse Robot Path Planning - Web Application
 * Backend: Java Built-in HTTP Server + Space-Time A* Planner
 * Frontend: Embedded HTML5 Canvas + Real-time REST API Polling
*
 * NEW: Keyboard shortcuts to replace (relocate) robots
 *   R            -> move all robots to random free cells
 *   H            -> send all robots back to their home charging docks
 *   1 / 2 / 3    -> select Robot 0 / 1 / 2
 *   Shift+Click -> move the selected robot to the clicked cell
 */
public class WebWarehousePlanner {


    public static final int PORT = 8080;
    public static final int GRID_WIDTH = 28;
    public static final int GRID_HEIGHT = 18;

    // Cell types
    public static final int EMPTY = 0;
    public static final int SHELF = 1;
    public static final int PICK_STATION = 2;
    public static final int DROP_STATION = 3;
    public static final int CHARGING_DOCK = 4;
    public static final int DYNAMIC_OBSTACLE = 5;


    // State
    private static final int[][] grid = new int[GRID_HEIGHT][GRID_WIDTH];
    private static final List<Point> pickStations = new ArrayList<>();
    private static final List<Point> dropStations = new ArrayList<>();
    private static final List<Point> chargingDocks = new ArrayList<>();
    private static final List<Robot> robots = new CopyOnWriteArrayList<>();
    private static final Map<String, Integer> vertexReservations = new ConcurrentHashMap<>();
    private static final Map<String, Integer> edgeReservations = new ConcurrentHashMap<>();


    private static volatile boolean isRunning = true;
    private static final ScheduledExecutorService simExecutor =
        Executors.newSingleThreadScheduledExecutor();

    public static void main(String[] args) throws IOException {
        initializeLayout();
        initializeRobots();

        //Start background simulation tick (every 250ms)
        simExecutor.scheduleAtFixedRate(WebWarehousePlanner::simulationStep, 0, 250,
            TimeUnit.MILLISECONDS);

        // Start Web Server
        HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
        server.createContext("/", new StaticPageHandler());
        server.createContext("/api/state", new StateHandler());
        server.createContext("/api/dispatch", new DispatchHandler());
        server.createContext("/api/obstacle", new ObstacleHandler());
        server.createContext("/api/clear-obstacles", new ClearObstaclesHandler());
        server.createContext("/api/toggle-sim", new ToggleSimHandler());

        server.createContext("/api/relocate", new RelocateHandler());   // NEW
```

```
        server.setExecutor(Executors.newFixedThreadPool(4));
        server.start();
66
67
68
69          System.out.println("============================================================");
70          System.out.println(" Warehouse Robot Path Planner Web Server is LIVE!");
71          System.out.println(" Open your browser: http://localhost:" + PORT);
72          System.out.println(" Press Ctrl + C in terminal to stop.");
73          System.out.println("============================================================");
74      }
75
76      // ================================================================================
77      //1.  WAREHOUSE INITIALIZATION
78      // ================================================================================
79      privatestatic void initializeLayout() {
80          chargingDocks.add(new Point(1, 1));
81          chargingDocks.add(new Point(1, 16));
82          chargingDocks.add(new Point(26, 1));
83          chargingDocks.add(new Point(26, 16));
84          for(Point p : chargingDocks) grid[p.y][p.x] = CHARGING_DOCK;
85
86          pickStations.add(new Point(4, 0));
87          pickStations.add(new Point(12, 0));
88          pickStations.add(new Point(20, 0));
89          for (Point p : pickStations) grid[p.y][p.x] = PICK_STATION;
90
91          dropStations.add(new Point(4, 17));
92          dropStations.add(new Point(12, 17));
93          dropStations.add(new Point(20, 17));
94          for (Point p : dropStations) grid[p.y][p.x] = DROP_STATION;
95
96          int[][]shelfXBlocks={{4,7}, {10, 13}, {16, 19}, {22, 25}};
97          int[][]shelfYBlocks={{3,6}, {9, 14}};
98          for(int[]xRange:shelfXBlocks) {
99              for(int[]yRange:shelfYBlocks) {
100                 for(intx=xRange[0]; x <= xRange[1]; x++) {
101                     for(inty=yRange[0]; y <= yRange[1]; y++) {
102                         grid[y][x] = SHELF;
103                     }
104                 }
105             }
106         }
107     }
108
109     private static void initializeRobots() {
110         robots.add(new Robot(0, new Point(1, 1), "#6F42C1"));
111         robots.add(new Robot(1, new Point(1, 16), "#D63384"));
112         robots.add(new Robot(2, new Point(26, 1), "#0DCAF0"));
113     }
114
115     // ================================================================================
116     //2.  SPACE-TIME A* ALGORITHM
117     // ================================================================================
118     staticclassNode implements Comparable<Node> {
119         int x, y, t;
120         double g, f;
121         Node parent;
122
123         public Node(int x, int y, int t, double g, double f, Node parent) {
124             this.x = x; this.y = y; this.t = t; this.g = g; this.f = f; this.parent = parent;
125         }
126
127         @Override
128         public int compareTo(Node o) {
129             return Double.compare(this.f, o.f);
130         }
131     }
132
133     private static synchronized List<Point> findSpaceTimePath(Point start, Point goal) {
134         int maxTime = 120;
135         PriorityQueue<Node> openSet = new PriorityQueue<>();
```

```
        Set<String> visited = new HashSet<>();


        double initialH = Math.abs(start.x - goal.x) + Math.abs(start.y - goal.y);
        openSet.add(new Node(start.x, start.y, 0, 0, initialH, null));

        int[][] moves = {{0, -1}, {0, 1}, {-1, 0}, {1, 0}, {0, 0}};

        while(!openSet.isEmpty()) {
            Node current = openSet.poll();

            if(current.x==goal.x&&current.y == goal.y) {
                List<Point> path = new ArrayList<>();
                Node curr = current;
                while (curr != null) {
                    path.add(0,newPoint(curr.x, curr.y));
                    curr = curr.parent;
                }
                return path;
            }

            if (current.t >= maxTime) continue;

            String stateKey = current.x + "," + current.y + "," + current.t;
            if (visited.contains(stateKey)) continue;
            visited.add(stateKey);

            int nextT = current.t + 1;

            for(int[] m : moves) {
                int nx = current.x + m[0];
                int ny = current.y + m[1];

                if (nx < 0 || nx >= GRID_WIDTH || ny < 0 || ny >= GRID_HEIGHT) continue;
                if (grid[ny][nx] == SHELF || grid[ny][nx] == DYNAMIC_OBSTACLE) continue;

                // Vertex conflict check
                if (vertexReservations.containsKey(nx + "," + ny + "," + nextT)) continue;

                //Edge/swap conflict check
                if(edgeReservations.containsKey(current.x + "," + current.y + "," + nx + ","
                    +ny+"," + nextT)) continue;

                double stepCost = (m[0] == 0 && m[1] == 0) ? 1.1 : 1.0;
                double ng = current.g + stepCost;
                double nh = Math.abs(nx - goal.x) + Math.abs(ny - goal.y);

                openSet.add(new Node(nx, ny, nextT, ng, ng + nh, current));
            }
        }
        return null;
    }

    private static synchronized void rebuildReservations() {
        vertexReservations.clear();
        edgeReservations.clear();

        for(Robot r : robots) {
            if(r.path == null || r.path.isEmpty()) continue;
            for(int t = r.pathIndex; t < r.path.size(); t++) {
                int relT = t - r.pathIndex;
                Point p = r.path.get(t);
                vertexReservations.put(p.x + "," + p.y + "," + relT, r.id);

                if(t>r.pathIndex) {
                    Pointprev = r.path.get(t - 1);
                    edgeReservations.put(prev.x + "," + prev.y + "," + p.x + "," + p.y + ","
                        +relT, r.id);
                }
            }
        }
```

APPENDIX L: Complete Source Code (continued)

```
204     }
205
206     privatestatic synchronized boolean planRobot(Robot robot, Point destination) {
207         rebuildReservations();
208         List<Point> path = findSpaceTimePath(robot.pos, destination);
209         if(path != null) {
210             robot.path = path;
211             robot.pathIndex = 0;
212             robot.target = destination;
213             rebuildReservations();
214             return true;
215
216         }
217         return false;
218     }
219
220     // ================================================================================
221     //3.  BACKGROUNDSIMULATION TICK
222     // ================================================================================
223     //NOTE:nowsynchronized so that relocating robots never clashes with a tick.
224     privatestaticsynchronized void simulationStep() {
225         if (!isRunning) return;
226         Random rand = new Random();
227
228
229         for(Robot robot : robots) {
230             if(!robot.state.equals("IDLE")) {
231                 robot.battery = Math.max(0.0, robot.battery - 0.08);
232             }
233
234             if(robot.waitTimer > 0) {
235             robot.waitTimer--;
236             continue;
237             }
238
239             if(robot.path != null && robot.pathIndex < robot.path.size() - 1) {
240                 Point nextStep = robot.path.get(robot.pathIndex + 1);
241
242                 if(grid[nextStep.y][nextStep.x] == DYNAMIC_OBSTACLE) {
243                     robot.replans++;
244                     planRobot(robot, robot.target);
245                     continue;
246                 }
247
248                 robot.pathIndex++;
249                 robot.pos = nextStep;
250                 robot.distanceTraveled++;
251
252             }else if (robot.path != null && robot.pathIndex >= robot.path.size() - 1) {
253                 robot.path = null;
254                 robot.pathIndex = 0;
255
256                 switch(robot.state) {
257                     case "TO_PICK":
258                         robot.state = "PICKING";
259                         robot.waitTimer = 3;
260                         break;
261                     case "PICKING":
262                         robot.state = "TO_DROP";
263                         planRobot(robot, dropStations.get(rand.nextInt(dropStations.size())));
264                         break;
265                     case "TO_DROP":
266                         robot.state = "DROPPING";
267                         robot.waitTimer = 2;
268                         break;
269                     case "DROPPING":
270                         robot.state = "RETURNING";
271                         planRobot(robot, robot.homeDock);
272                         break;
273                     case "RETURNING":
                        robot.state = "IDLE";
                        break;
```

APPENDIX L: Complete Source Code (continued)

```
                }
            } else {
                if(robot.pos.equals(robot.homeDock)) {
                    robot.battery = Math.min(100.0, robot.battery + 0.4);
                }
            }
        }
        rebuildReservations();
    }

    // ================================================================================
    //4.  HTTP API HANDLERS
    // ================================================================================
    staticclass StateHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            StringBuilder json = new StringBuilder("{");
            json.append("\"running\":").append(isRunning).append(",");

            // Grid
            json.append("\"grid\":[");
            for(inty=0;y<GRID_HEIGHT; y++) {
                json.append("[");
                for(intx=0;x<GRID_WIDTH; x++) {
                    json.append(grid[y][x]).append(x == GRID_WIDTH - 1 ? "" : ",");
                }
                json.append("]").append(y == GRID_HEIGHT - 1 ? "" : ",");
            }
            json.append("],");

            // Robots
            json.append("\"robots\":[");
            for(inti=0;i<robots.size(); i++) {
                Robot r = robots.get(i);
                //Localcopiessoa     relocate (path = null) cannot cause a
                    NullPointerException
                List<Point> path = r.path;
                int pathIndex = r.pathIndex;
                Point pos = r.pos;

                json.append("{")
                    .append("\"id\":").append(r.id).append(",")
                    .append("\"x\":").append(pos.x).append(",")
                    .append("\"y\":").append(pos.y).append(",")
                    .append("\"color\":\"").append(r.color).append("\",")
                    .append("\"state\":\"").append(r.state).append("\",")
                    .append("\"battery\":").append(String.format(Locale.US, "%.1f",
                        r.battery)).append(",")
                    .append("\"distance\":").append(r.distanceTraveled).append(",")
                    .append("\"replans\":").append(r.replans).append(",")
                    .append("\"path\":[");

                if (path != null) {
                    for(intp=pathIndex; p < path.size(); p++) {
                        Point pt = path.get(p);
                        json.append("[").append(pt.x).append(",").append(pt.y).append("]")
                            .append(p == path.size() - 1 ? "" : ",");
                    }
                }
                json.append("]}").append(i == robots.size() - 1 ? "" : ",");
            }
            json.append("]}");

            byte[] bytes = json.toString().getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set("Content-Type", "application/json");
            exchange.sendResponseHeaders(200, bytes.length);
            try (OutputStream os = exchange.getResponseBody()) { os.write(bytes); }
        }
    }
```

APPENDIX L: Complete Source Code (continued)

```
staticclassDispatchHandler implements HttpHandler {
    @Override
    publicvoidhandle(HttpExchange exchange) throws IOException {
        Randomrand=new Random();
        for(Robotrobot : robots) {
            if(robot.state.equals("IDLE") || robot.state.equals("RETURNING")) {
                robot.state = "TO_PICK";
                if (!planRobot(robot,
                    pickStations.get(rand.nextInt(pickStations.size())))) {
                    robot.state = "IDLE";
                }
            }
        }
        sendOk(exchange);
    }
}

staticclassObstacleHandler implements HttpHandler {
    @Override
    publicvoidhandle(HttpExchange exchange) throws IOException {
        String query = exchange.getRequestURI().getQuery();
        if(query!=null) {
            int  x  =  Integer.parseInt(query.split("&")[0].split("=")[1]);
            int y = Integer.parseInt(query.split("&")[1].split("=")[1]);

            if(x>=0&&x<GRID_WIDTH&&y>= 0 && y < GRID_HEIGHT) {
                if (grid[y][x] == EMPTY) {
                    grid[y][x] = DYNAMIC_OBSTACLE;
                    Pointblocked  =newPoint(x, y);
                    for (Robot r : robots) {
                        if (r.path != null && r.path.subList(r.pathIndex,
                            r.path.size()).contains(blocked)) {
                            r.replans++;
                            planRobot(r, r.target);
                        }
                    }
                }elseif(grid[y][x]==DYNAMIC_OBSTACLE) {
                    grid[y][x] = EMPTY;
                }
            }
        }
        sendOk(exchange);
    }
}

staticclassClearObstaclesHandlerimplements HttpHandler {
    @Override
    publicvoidhandle(HttpExchangeexchange) throws IOException {
        for(inty=0;y<GRID_HEIGHT; y++) {
            for(intx=0;x<GRID_WIDTH; x++) {
                if(grid[y][x]==   DYNAMIC_OBSTACLE) grid[y][x] = EMPTY;
            }
        }
        sendOk(exchange);
    }
}

staticclass ToggleSimHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange) throws IOException {
        isRunning = !isRunning;
        sendOk(exchange);
    }
}

// ---------------------------------------------------------------------
// NEW: Relocate (replace) robots
//   /api/relocate?mode=random        -> all robots to random empty cells
//   /api/relocate?mode=home          -> all robots back to their home docks
//   /api/relocate?id=0&x=5&y=2       -> one robot to a chosen cell
```

APPENDIX L: Complete Source Code (continued)

```
    // ---------------------------------------------------------------------
    staticclassRelocateHandlerimplements HttpHandler {
        @Override
        publicvoidhandle(HttpExchangeexchange) throws IOException {
            Map<String,String>params=new HashMap<>();
            String query = exchange.getRequestURI().getQuery();
            if (query != null) {
                  for(Stringpair:query.split("&")) {
                    String[] kv = pair.split("=");
                    if(kv.length==   2)params.put(kv[0], kv[1]);
                }
            }

            synchronized(WebWarehousePlanner.class) {
                try{
                    String mode = params.get("mode");
                    if("random".equals(mode)) {
                    relocateAllRandom();
                    }elseif("home".equals(mode)) {
                        for(Robotr   :robots) placeRobot(r, new Point(r.homeDock.x,
                            r.homeDock.y));
                    }elseif(params.containsKey("id")) {
                        int  id  =  Integer.parseInt(params.get("id"));
                        int  x  =  Integer.parseInt(params.get("x"));
                        int y = Integer.parseInt(params.get("y"));
                          for(Robotr :robots) {
                             if(r.id ==id && isFreeCell(x, y, r)) {
                                placeRobot(r, new Point(x, y));
                             }
                        }
                    }
                }catch(NumberFormatException ignored) {
                    //badqueryvaluesare ignored
                }
                rebuildReservations();
            }
            sendOk(exchange);
        }
    }

    /**Puts a robot on a cell and cancels whatever it was doing. */
    private static void placeRobot(Robot r, Point p) {
        r.pos = p;
        r.path   =   null;
        r.pathIndex  =  0;
        r.target  =  null;
        r.waitTimer  =  0;
        r.state = "IDLE";
    }

    /**A  cellisvalidifit is inside the grid, not a shelf/obstacle and no other robot is
        on it. */
    privatestaticbooleanisFreeCell(int x, int y, Robot self) {
        if(x<0||x>=GRID_WIDTH  ||  y  <  0  ||  y  >=  GRID_HEIGHT)  return  false;
        if(grid[y][x]==SHELF  ||  grid[y][x]  ==  DYNAMIC_OBSTACLE)  return  false;
        for(Robotother:robots) {
            if(other!=self && other.pos.x == x && other.pos.y == y) return false;
        }
        return true;
    }

    privatestaticvoidrelocateAllRandom() {
        List<Point> freeCells = new ArrayList<>();
        for(inty=0;y<GRID_HEIGHT; y++) {
            for(intx=0;x<GRID_WIDTH; x++) {
                if(grid[y][x]==EMPTY) freeCells.add(new Point(x, y));
            }
        }
        Collections.shuffle(freeCells);
        int i = 0;
```

```
        for(Robot r : robots) {

            placeRobot(r, freeCells.get(i++));   // every robot gets a different cell
        }
    }

    private static void sendOk(HttpExchange exchange) throws IOException {
    byte[] b = "{\"status\":\"ok\"}".getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().set("Content-Type", "application/json");
    exchange.sendResponseHeaders(200, b.length);
    try (OutputStream os = exchange.getResponseBody()) { os.write(b); }
    }

    // =====================================================================
    // 5. EMBEDDED FRONTEND (HTML5 / JS / Canvas)
    // =====================================================================
    static class StaticPageHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String html = "<!DOCTYPE html>\n" +
                "<html lang=\"en\">\n" +
                "<head>\n" +
                " <metacharset=\"UTF-8\"/>\n"+
                " <title>Autonomous Warehouse Robot Path Planning</title>\n" +
                " <style>\n"+
                "    body{font-family:'SegoeUI',system-ui,sans-serif;margin:0;
                    background: #0F172A; color: #F8FAFC; display: flex; height: 100vh; }\n" +
                "    #main { flex: 1; padding: 20px; display: flex; flex-direction: column;
                    align-items: center; justify-content: center; }\n" +
                "    #sidebar { width: 340px; background: #1E293B; border-left: 1px solid
                    #334155; padding: 20px; box-sizing: border-box; overflow-y: auto; }\n" +
                "    canvas { background: #FFFFFF; border-radius: 8px; box-shadow: 0 10px
                    25px rgba(0,0,0,0.5); cursor: pointer; }\n" +
                "    .btn { display: block; width: 100%; padding: 12px; margin-bottom: 10px;
                    border: none; border-radius: 6px; font-weight: 600; font-size: 14px;
                    cursor: pointer; transition: 0.2s; }\n" +
                "    .btn-green { background: #10B981; color: white; }\n" + .btn-blue
                "    { background: #3B82F6; color: white; }\n" +
                "    .btn-gray { background: #475569; color: white; }\n" + .btn-
                "    purple { background: #8B5CF6; color: white; }\n" +
                "    .card { background: #334155; padding: 12px; border-radius: 6px;
                    margin-bottom: 10px; font-size: 13px; line-height: 1.5; border-left: 4px
                    solid #60A5FA; }\n" +
                "    .badge { display: inline-block; padding: 2px 6px; border-radius: 4px;
                    font-size: 11px; font-weight: bold; }\n" +
                "    .keys { background: #0F172A; border: 1px solid #475569; border-radius:
                    6px; padding: 10px; font-size: 12px; line-height: 1.7; color: #CBD5E1;
                    }\n" +
                "    kbd { background: #475569; border-radius: 4px; padding: 1px 6px;
                    font-weight: bold; color: #FACC15; }\n" +
                "    h2 { font-size: 18px; margin-top: 0; border-bottom: 1px solid #475569;
                    padding-bottom: 8px; }\n" +
                " </style>\n"+
                "</head>\n" +
                "<body>\n" +
                " <divid=\"main\">\n"+
                "    <h1style=\"margin-top:0;font-size:22px;\">AutonomousWarehouseAMR
                    Fleet (Space-Time A*)</h1>\n" +
                "    <canvas id=\"mapCanvas\" width=\"980\" height=\"630\"></canvas>\n" +
                "    <p style=\"color: #94A3B8; font-size: 13px;\">\uD83D\uDCA1 <b>Click any
                    empty aisle</b> to drop a dynamic obstacle (pallet/worker) and watch
                    real-time replanning.</p>\n" +
                "  </div>\n" +
                "  <div id=\"sidebar\">\n" +
                "    <h2>Control Center</h2>\n" +
                "    <button id=\"btnToggle\" class=\"btn btn-green\"
                    onclick=\"toggleSim()\">\u23F8 Pause Simulation</button>\n" +
                "    <button class=\"btn btn-blue\" onclick=\"dispatchMissions()\">\u26A1
                    Dispatch New Missions</button>\n" +
                "    <button class=\"btn btn-purple\"
                    onclick=\"relocate('mode=random')\">\uD83D\uDD00 Replace Robots Randomly
```

APPENDIX L: Complete Source Code (continued)

```
    (R)</button>\n" +
"    <button class=\"btn btn-gray\"
    onclick=\"relocate('mode=home')\">\uD83C\uDFE0 Send Robots Home
    (H)</button>\n" +
"    <button class=\"btn btn-gray\" onclick=\"clearObstacles()\">\uD83E\uDDF9
    Clear Dynamic Obstacles</button>\n" +
"    <h2 style=\"margin-top:20px;\">Keyboard Shortcuts</h2>\n" +
"    <div class=\"keys\">\n" +
"      <kbd>R</kbd> Replace all robots randomly<br>\n" +
"      <kbd>H</kbd> Send all robots home<br>\n" +
"      <kbd>1</kbd><kbd>2</kbd><kbd>3</kbd>SelectRobot0/1/2<br>\n" +
"      <kbd>Shift</kbd> + Click Move selected robot there<br>\n" +
"      <kbd>Space</kbd> Pause / Resume<br>\n" +
"      <kbd>D</kbd> Dispatch missions\n" +
"
"    </div>\n" +
"    <h2 style=\"margin-top:20px;\">Live Fleet Telemetry</h2>\n" +
"    <div id=\"telemetry\"></div>\n" +
"  </div>\n" +
"  <script>\n" +
"    const canvas = document.getElementById('mapCanvas');\n" +
"    const ctx = canvas.getContext('2d');\n" +
"    const CELL = 35;\n" +
    const COLORS = { 0:'#F8F9FA', 1:'#334155', 2:'#10B981', 3:'#F97316',
    4:'#3B82F6', 5:'#EF4444' };\n" +
"    letstate=null;\n"+
"    letselectedRobot=0;\n"+
"\n" +
"    asyncfunctionfetchState(){\n"+
"      try{\n"+
"        constres= awaitfetch('/api/state');\n"+
"        state=awaitres.json();\n"+
"        document.getElementById('btnToggle').innerText=state.running ?
    '\u23F8 Pause Simulation' : '\u25B6 Resume Simulation';\n" +
"        document.getElementById('btnToggle').className=state.running ?
    'btn btn-gray' : 'btn btn-green';\n" +
"        render();\n" +
"        renderTelemetry();\n"+
"      }catch(e){}\n"+
"    }\n"+
"\n" +
"    functionrender(){\n"+
"      if(!state)return;\n"+
"      ctx.clearRect(0,0,canvas.width,canvas.height);\n"+
"      for(lety=0;y<18;y++){\n"+
"        for(letx= 0;x<28;x++){\n"+
"          consttype =state.grid[y][x];\n"+
"          ctx.fillStyle=COLORS[type];\n"+
"          ctx.fillRect(x*CELL,y*CELL,CELL,CELL);\n"+
"          ctx.strokeStyle='#E2E8F0';\n"+
"          ctx.strokeRect(x*CELL,y*CELL,CELL,CELL);\n"+
"          if(type===2)drawLabel(x,y,'PICK','#FFF',10);\n"+
"          elseif(type===3)drawLabel(x,y,'DROP','#FFF',10);\n" +
"          elseif(type===4)drawLabel(x,y,'\u26A1','#FFF',14);\n" +
"          elseif(type===5)drawLabel(x,y,'\u26A0\uFE0F','#FFF',
    14);\n" +
"        }\n" +
"      }\n" +
"      state.robots.forEach(r => {\n" +
"        if (r.path && r.path.length > 1) {\n" +
"          ctx.strokeStyle = r.color;\n" +
"          ctx.setLineDash([4, 4]);\n" +
"          ctx.lineWidth = 3;\n" +
"          ctx.beginPath();\n" +
"          r.path.forEach((p, idx) => {\n" +
"
            constpx  =p[0]*CELL+CELL/2,py=p[1]*CELL+CELL /
    2;\n" +
"            idx === 0 ? ctx.moveTo(px, py) : ctx.lineTo(px, py);\n" +
"          });\n" +
"          ctx.stroke();\n" +
"          ctx.setLineDash([]);\n" +
```

APPENDIX L: Complete Source Code (continued)

```
"        }\n" +
"        const cx = r.x * CELL + CELL / 2, cy = r.y * CELL + CELL / 2;\n" +
"        ctx.fillStyle = r.color;\n" +
"        ctx.beginPath(); ctx.arc(cx, cy, 13, 0, Math.PI * 2); ctx.fill();\n"
    +
"        ctx.strokeStyle = '#000'; ctx.lineWidth = 2; ctx.stroke();\n" +
"        if (r.id === selectedRobot) {\n" +
"          ctx.strokeStyle = '#FACC15'; ctx.lineWidth = 3;\n" +
"          ctx.beginPath(); ctx.arc(cx, cy, 17, 0, Math.PI * 2);
    ctx.stroke();\n" +
"        }\n"+
"        ctx.fillStyle='#FFF';ctx.font='bold10pxArial';\n"+
"        ctx.textAlign='center';ctx.textBaseline='middle';\n"+
"        ctx.fillText('R'+r.id,cx,cy);\n"+
"      });\n"+
"    }\n"+
"\n" +
"    functiondrawLabel(x,y,txt,color,size){\n"+
"      ctx.fillStyle =color;ctx.font='bold'+size+'pxArial';\n"+
"      ctx.textAlign ='center';ctx.textBaseline='middle';\n"+
"      ctx.fillText(txt,x*CELL+CELL/2,y*CELL+CELL/2);\n"+
"    }\n"+
"\n" +
"    functionrenderTelemetry(){\n"+
"      lethtml='';\n"+
"      state.robots.forEach(r=>{\n"+
"                  consтsel= (r.id===selectedRobot)?'<spanclass=\"badge\"
     style=\"background:#FACC15;color:#000\">SELECTED</span>' : '';\n" +
"        html += `<div class=\"card\"
    style=\"border-left-color:${r.color}\">` +\n" +
"          `<b>Robot #${r.id}</b> | <span class=\"badge\"
    style=\"background:${r.color}\">${r.state}</span>${sel}<br>` +\n" +
"          `Battery: ${r.battery}%|Pos:(${r.x},${r.y})<br>`+\n"+
"          `Distance: ${r.distance}cells|Replans:${r.replans}</div>`;\n" +
"      });\n"+
"      document.getElementById('telemetry').innerHTML=html;\n"+
"    }\n"+
"\n" +
"    canvas.addEventListener('click',e=>{\n"+
"      constrect=canvas.getBoundingClientRect();\n"+
"      constx=Math.floor((e.clientX-rect.left)/CELL);\n"+
"      consty=Math.floor((e.clientY-rect.top)/CELL);\n"+
"      if(e.shiftKey){\n"+
"        relocate(`id=${selectedRobot}&x=${x}&y=${y}`);\n"+
"        return;\n"+
"      }\n"+
"      fetch(`/api/obstacle?x=${x}&y=${y}`,{method:'POST'}).then(()=>
    fetchState());\n" +
"    });\n"+
"\n" +
"    document.addEventListener('keydown',e=>{\n"+
"      if(e.ctrlKey ||e.metaKey||e.altKey)return;\n"+
"      constk=e.key.toLowerCase();\n"+
"      if(k==='r') relocate('mode=random');\n"+
"      elseif(k=== 'h')relocate('mode=home');\n"+
"      elseif(k=== 'd')dispatchMissions();\n"+
"      elseif(k=== ''){e.preventDefault();toggleSim();}\n"+
"      elseif(k>= '1'&&k<='9'){selectedRobot=parseInt(k)-1;
    fetchState(); }\n" +
"    });\n"+
"\n" +
"    functionrelocate(query){fetch('/api/relocate?'+query,{method:
    'POST' }).then(() => fetchState()); }\n" +
"    function toggleSim() { fetch('/api/toggle-sim', { method: 'POST'
    }).then(() => fetchState()); }\n" +
"    function dispatchMissions() { fetch('/api/dispatch', { method: 'POST'
    }).then(() => fetchState()); }\n" +
"    function clearObstacles() { fetch('/api/clear-obstacles', { method:
    'POST' }).then(() => fetchState()); }\n" +
"    setInterval(fetchState, 200);\n" +
```

APPENDIX L: Complete Source Code (continued)

```
                "  </script>\n" +
                "</body>\n" +
                "</html>";

            byte[] b = html.getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
            exchange.sendResponseHeaders(200, b.length);
            try (OutputStream os = exchange.getResponseBody()) { os.write(b); }
        }
    }

    // ================================================================
    // 6. MODELS
    // ================================================================
    staticclass Point {
        intx, y;
        public Point(int x, int y) { this.x = x; this.y = y; }
        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Point)) return false;
            Point p = (Point) o;
            return x == p.x && y == p.y;
        }
        @Override
        public int hashCode() { return Objects.hash(x, y); }
    }

    static class Robot {
        int id;
        Point pos;
        Point homeDock;
        String color;
        String state = "IDLE";
        double battery = 100.0;
        List<Point> path = null;
        int pathIndex = 0;
        Point target = null;
        int waitTimer = 0;
        int distanceTraveled = 0;
        int replans = 0;


        public Robot(int id, Point startPos, String color) {
            this.id = id; this.pos = startPos; this.homeDock = startPos; this.color = color;
        }
    }
}
```